
Documentação de Projeto

para o sistema

Cognita

Versão 8.0

Projeto de sistema elaborado pelo aluno Lucas Hemétrio Teixeira e apresentado ao curso de **Engenharia de Software** da **PUC Minas** como parte do Trabalho de Conclusão de Curso (TCC) sob orientação de conteúdo dos professores Danilo de Quadros Maia Filho, Leonardo Vilela Cardoso, Raphael Ramos Dias Costa, orientação acadêmica do professor Cleiton Silva Tavares e orientação de TCC II do professor Ramon Lacerda Marques.

28/05/2026

Tabela de Conteúdo

Tabela de Conteúdo	ii
Histórico de Revisões	ii
1. Introdução	1
2. Modelos de Usuário e Requisitos	1
2.1 Descrição de Atores	1
2.2 Modelos de Usuários	2
2.3 Modelo de Casos de Uso e Histórias de Usuários	3
2.4 Diagrama de Sequência do Sistema e Contrato de Operações	6
3. Modelos de Projeto	10
3.1 Diagrama de Classes	10
3.2 Diagramas de Sequência	12
3.3 Diagramas de Comunicação	14
3.4 Arquitetura	18
3.5 Diagramas de Estados	20
3.6 Diagrama de Componentes e Implantação	21
4. Projeto de Interface com Usuário	24
4.1 Esboço das Interfaces Comuns a Todos os Atores	25
5. Glossário e Modelos de Dados	30
5.1 Glossário	30
5.2 Modelo de Dados – Property Graph Model	32
6. Casos de Teste	33
6.1 Plano de Teste de Aceitação	34
6.2 Plano de Teste de Integração	35
6.3 Avaliação da Qualidade do Grafo de Conhecimento	36
7. Cronograma e Processo de Implementação	37
7.1 Cronograma do Projeto	37
7.2 Processo de Implementação	39
8. Post-Mortem	39
8.1 Conquistas e Aprendizados	40
8.2 Experiências Negativas e Limitações	40
8.3 Segurança, Conformidade e Restrições	41
8.4 Integração com Serviços Externos	41
8.5 Lições Aprendidas e Próximos Passos	41
8.6 Melhoria Contínua / Recomendações Futuras	42
9. Conclusão	42

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Atividade 3 - TCC 1	24/09/25	Introdução, Persona, Casos de Uso, Histórias de Usuário, Protótipos Iniciais	1.0

Atividade 4 - TCC 1	13/10/25	Correções/Ajustes Atividade 3, Diagramas de Sequência do Sistema, Diagrama de Classes, Diagrama de Comunicação	2.0
Atividade 5 - TCC 1	02/11/25	Correções/Ajustes Atividade 4, Diagrama de Arquitetura, Diagrama de Estados, Diagrama de Componentes, Diagrama de Implantação, Glossário, Property Graph Model	3.0
Atividade 6 - TCC 1	16/11/25	Correções/Ajustes Atividade 5, Cronograma	4.0
Atividade 7 - TCC 1	30/11/25	Correções/Ajustes Atividade 6, Casos de Testes	5.0
Atividade 8 - TCC 1	14/12/25	Adicionar teste relacionado a IA	6.0
Atividade 3 - TCC 2	26/04/26	Correção atualizando serviços externos	7.0
Atividade 4 - TCC 2	28/05/26	Atualização da seção 7.2 (Processo de Implantação) e adição da seção 8 (Post-Mortem) com subseções de análise crítica. E refinamento geral.	8.0

1. Introdução

Este documento agrega a elaboração e a revisão dos modelos de domínio e de projeto para o sistema Cognita. Ele serve como o *blueprint* técnico e funcional que guiará o desenvolvimento do *software*, detalhando sua arquitetura, comportamento e estrutura de dados.

O sistema Cognita nasce para solucionar uma lacuna fundamental no ecossistema de ferramentas de estudo digital: a fragmentação entre aplicativos de anotação de alta fidelidade, que tratam documentos como objetos isolados, e as plataformas de Gestão de Conhecimento Pessoal (PKM - *Personal Knowledge Management*), que, embora foquem na conexão de ideias, oferecem uma experiência básica de interação com os materiais-fonte.

A proposta central do Cognita é unificar esses dois mundos, criando uma plataforma coesa para o estudo ativo. O objetivo é transformar um repositório passivo de informações em uma base de conhecimento dinâmica e interconectada, onde a inovação chave reside na construção de um grafo de conhecimento de forma automática, utilizando técnicas de Processamento de Linguagem Natural (PLN) para identificar conceitos no texto e análise de co-ocorrência para revelar as conexões entre eles.

A referência principal para a descrição geral do problema, domínio, escopo e requisitos do sistema é o Documento de Visão, que acompanha este documento. Anexo a este documento também se encontra o Glossário, que define os termos técnicos e conceituais utilizados ao longo do projeto.

2. Modelos de Usuário e Requisitos

Esta seção é dedicada à modelagem dos usuários e dos requisitos funcionais do sistema Cognita. O objetivo é definir quem são os usuários, quais são seus objetivos e como eles irão interagir com a plataforma para alcançá-los. A análise começa com a identificação e descrição dos atores do sistema, seguida pela elaboração de modelos de usuário mais detalhados, como personas, para aprofundar o entendimento de suas necessidades e contextos. Por fim, os requisitos são formalizados através de casos de uso e histórias de usuário, que descrevem as funcionalidades do sistema sob a perspectiva do usuário final.

2.1 Descrição de Atores

Esta subseção descreve o ator que interage com o sistema. O Cognita foi modelado em torno de um único ator principal, que reúne os diferentes perfis de usuário contemplados pela plataforma.

- Ator: Estudante / Pesquisador:
 - Descrição: Este ator representa o usuário final do sistema, englobando um espectro de aprendizes sérios, como estudantes universitários (graduação e pós-graduação),

pesquisadores acadêmicos, concurseiros e autodidatas. A característica principal deste ator é a necessidade de gerenciar, analisar e sintetizar um material de estudo complexo, predominantemente em formato de texto (artigos em PDF, livros digitais, anotações).

- **Objetivos no Sistema:** O objetivo primário do Estudante / Pesquisador é transformar sua biblioteca digital de um repositório estático para uma base de conhecimento ativa e interconectada. Ele busca uma ferramenta que não apenas centralize seus materiais, mas que o auxilie ativamente na descoberta de conexões conceituais latentes, na identificação de temas centrais e na visualização da estrutura de seu próprio conhecimento.
- **Interações com o Sistema:** As principais interações deste ator com o Cognita incluem: realizar o upload e gerenciamento de documentos PDF; criar anotações manuscritas e digitadas diretamente sobre os documentos e em notas independentes; utilizar a busca global para encontrar informações em toda a sua base de conhecimento; e, crucialmente, explorar o grafo de conhecimento gerado automaticamente para navegar e compreender as relações entre os diferentes tópicos de estudo.

2.2 Modelos de Usuários

Para aprofundar a compreensão sobre o ator "Estudante / Pesquisador", foi desenvolvida uma persona, detalhada na Tabela 1. A decisão de focar em uma única persona é estratégica: o arquétipo "Ana Oliveira, a Mestranda Organizada" foi escolhido por representar o ponto de maior complexidade dentro do espectro do ator definido. Ela encapsula tanto as necessidades de organização e aprendizado de um estudante quanto as de síntese e descoberta de um pesquisador. Ao projetar o sistema para atender às suas necessidades, garantimos que as funcionalidades essenciais para outros usuários no espectro (como graduandos ou concurseiros) também sejam contempladas.

	Biografia e Comportamento Ana Oliveira é uma mestranda dedicada à História Contemporânea. Sua rotina é intensa, dividida entre aulas, grupos de estudo e, principalmente, a pesquisa para sua dissertação. Ela lida com um volume massivo de artigos acadêmicos, livros digitalizados e anotações de fontes primárias, todos em formato PDF. Ana é tecnologicamente adepta e valoriza ferramentas que otimizam seu tempo. Ela já tentou de tudo: usa o GoodNotes em seu tablet para a leitura e anotação inicial dos PDFs, pela excelente experiência com a caneta, mas depois se vê forçada a copiar e colar manualmente seus insights para o Obsidian, onde tenta construir um mapa de suas ideias. Ela se sente constantemente frustrada por esse
Idade: 28 anos	
Ocupação: Mestranda e Pesquisadora	

	fluxo de trabalho desconectado e pela sensação de que existem conexões importantes em sua pesquisa que ela ainda não conseguiu enxergar.
Objetivos - Centralizar todos os seus materiais de pesquisa em um único lugar. - Descobrir conexões inesperadas entre autores e conceitos de diferentes artigos. - Otimizar seu tempo de revisão, podendo pesquisar instantaneamente em todas as suas anotações, incluindo as manuscritas. - Sentir que tem o controle total sobre sua base de conhecimento, e não que suas ideias estão espalhadas em arquivos e pastas.	Frustrações - Ter que usar dois ou três aplicativos diferentes para completar um ciclo de estudo (leitura, anotação e conexão). - Perder tempo procurando a fonte original de uma anotação ou citação que fez semanas atrás. - Suas anotações manuscritas serem "dados mortos", impossíveis de serem pesquisados globalmente. - A visão em grafo do Obsidian ser útil, mas depender 100% de seu esforço manual para criar cada link.

Tabela 1. Persona Ana Oliveira

2.3 Modelo de Casos de Uso e Histórias de Usuários

Esta subseção detalha as interações funcionais entre o ator e o sistema Cognita. Primeiramente, é apresentado um modelo de casos de uso que descreve as principais funcionalidades do sistema de forma macro, conforme ilustrado na Figura 1. Em seguida, as histórias de usuário, detalhadas na Tabela 2, traduzem essas funcionalidades em requisitos granulares e centrados nas necessidades da persona definida.

2.3.1 Diagrama de Caso de Uso

O diagrama de casos de uso para o sistema Cognita, apresentado na Figura 1, é centrado no único ator definido, o Estudante / Pesquisador. Este ator interage com um conjunto de funcionalidades principais que representam o núcleo da proposta de valor da plataforma.

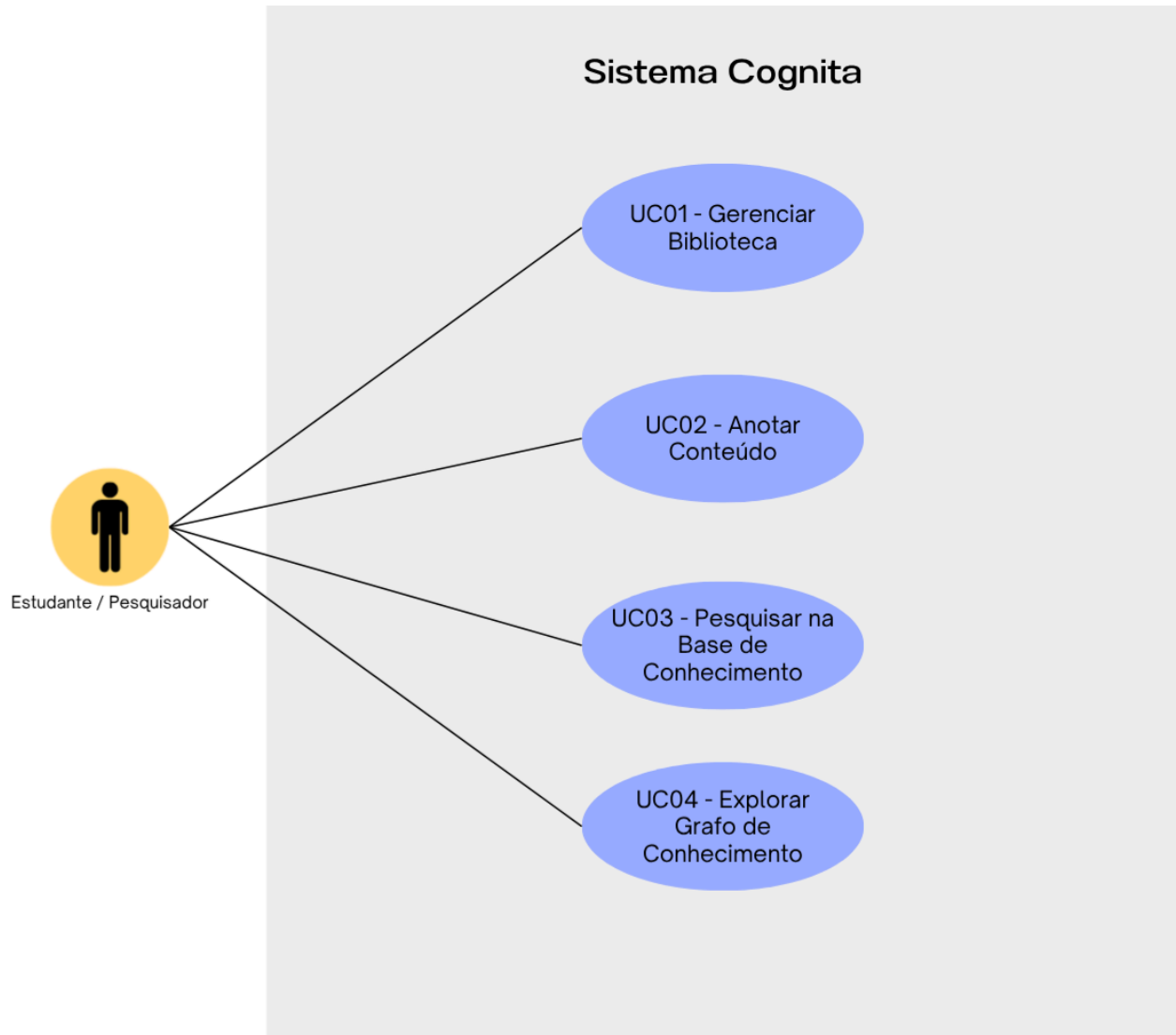


Figura 1. Diagrama de Caso de Uso

2.3.2 Histórias de Usuário

As histórias de usuário a seguir, apresentadas na Tabela 2, são formuladas a partir da perspectiva da persona "Ana Oliveira" e estão agrupadas por épicos que correspondem às funcionalidades centrais do sistema.

Caso de Uso	ID	Historia
-------------	----	----------

Gerenciar Biblioteca	US 01	Como Ana, a mestranda, eu quero fazer o upload de múltiplos artigos em PDF, para que eu possa adicionar todo o material de um novo tópico de pesquisa à minha biblioteca.
	US 02	Como Ana, eu quero organizar meus documentos em projetos, para que eu possa separar os materiais de diferentes matérias ou capítulos da minha dissertação.
Anotar Conteúdo	US 03	Como Ana, eu quero fazer anotações manuscritas diretamente nas margens de um PDF usando minha caneta <i>stylus</i> , para que eu possa capturar ideias da mesma forma que faria em papel.
	US 04	Como Ana, eu quero criar notas independentes que não estão atreladas a nenhum documento, para que eu possa registrar <i>brainstormings</i> ou ideias gerais.
	US 05	Como Ana, eu quero que minhas anotações sejam salvas de forma persistente, para que eu não perca meu trabalho.
Pesquisa na Base de Conhecimento	US 06	Como Ana, eu quero que o sistema processe automaticamente o texto de PDFs digitalizados e de minhas anotações manuscritas, para que todo o conteúdo da minha biblioteca se torne pesquisável.
	US 07	Como Ana, eu quero realizar uma busca por uma palavra-chave e ver resultados de todos os meus documentos e notas (digitadas e manuscritas), para que eu possa encontrar rapidamente qualquer informação que já estudei.
Explorar Grafo de Conhecimento	US 08	Como Ana, eu quero ter uma visão geral que me mostre os tópicos principais dos meus estudos e como eles se ligam, para que eu possa entender o quadro geral de um assunto complexo sem me perder nos detalhes.
	US 09	Como Ana, ao ver uma conexão entre dois tópicos, eu quero poder clicar nela e ser levada para a frase ou anotação exata em cada documento, para que eu possa lembrar e validar o contexto original daquela ligação.

Tabela 2. Histórias de Usuário

2.4 Diagrama de Sequência do Sistema e Contrato de Operações

Esta subseção apresenta os Diagramas de Sequência do Sistema (DSS) para os principais casos de uso, ilustrando a ordem cronológica das interações entre o ator "Estudante / Pesquisador" e o

sistema Cognita. Para cada operação de sistema iniciada pelo ator em um DSS, é detalhado um Contrato de Operação, que formaliza as responsabilidades do sistema, especificando as pré-condições necessárias para a operação e as pós-condições que garantem seu resultado.

Os diagramas a seguir detalham os fluxos de interação mais complexos e centrais para a proposta de valor do sistema Cognita, focando nos comportamentos assíncronos e nas interações ricas em dados. Fluxos de CRUD mais simples, como a criação de um novo projeto (US 02), foram omitidos por não apresentarem complexidade dinâmica significativa, estando sua lógica já representada no Diagrama de Classes. Da mesma forma, o fluxo para criar notas independentes (US 04) é coberto pelo diagrama de 'Fazer Anotação Manuscrita' (Figura 3), pois o comportamento dinâmico da interação é idêntico.

2.4.1 DSS – Realizar Upload de Documento

O diagrama apresentado na Figura 2, modela o fluxo de interação para as histórias de usuário US 01 e US 06. A modelagem é unificada em um único diagrama pois a ação do usuário de realizar o *upload* de um arquivo (*solicitarUpload*) é o gatilho para duas funcionalidades interligadas: a gestão do arquivo na biblioteca (atendendo à US 01) e o início do processamento assíncrono de conteúdo (OCR/HCR) pelo sistema para tornar esse arquivo pesquisável (atendendo à US 06). A Tabela 3 descreve o contrato do fluxo apresentado.

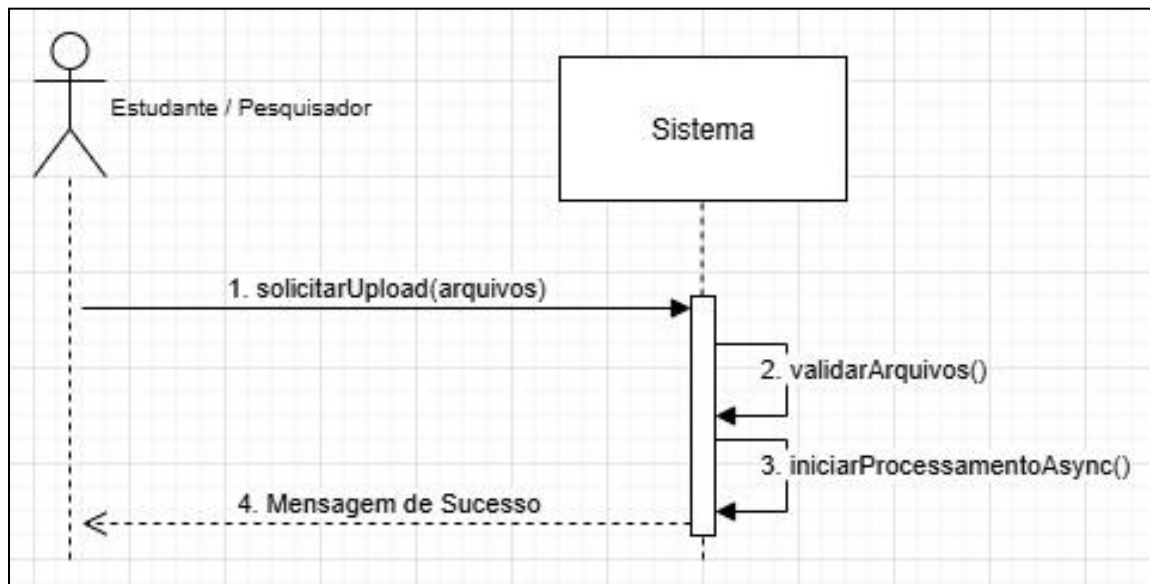


Figura 2. DSS para Upload de Documento

Contrato	<i>Upload</i> de Documento
Operação	<i>solicitarUpload</i> (arquivos)
Referências cruzadas	Histórias de Usuário: US 01, US 06
Pré-condições	O ator "Estudante / Pesquisador" deve estar autenticado no sistema
Pós-condições	O documento é adicionado à biblioteca do usuário com o estado "Processando".

	O sistema inicia o processamento do conteúdo do documento em segundo plano. O usuário recebe uma confirmação do <i>upload</i>.
--	--

Tabela 3. Contrato Upload de Documento

2.4.2 DSS – Fazer Anotação Manuscrita

Este diagrama, apresentado na Figura 3, modela o fluxo de interação para as histórias de usuário US 03 e US 05. A sequência de eventos de desenho representa a funcionalidade de anotação manuscrita (atendendo à US 03), enquanto a operação solicitarSalvar, acionada explicitamente pelo usuário ao concluir o trabalho, garante que a anotação seja persistida no servidor (atendendo à US 05). A Tabela 4 descreve o contrato do fluxo apresentado.

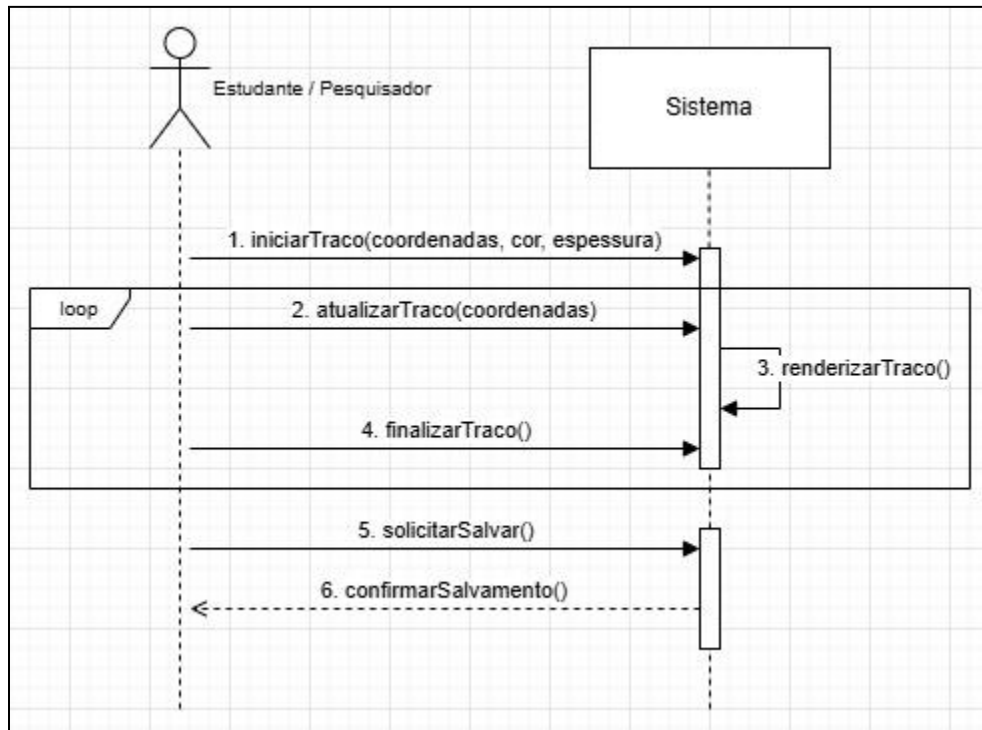


Figura 3. DSS para Fazer Anotação Manuscrita

Contrato	Anotação Manuscrita
Operação	iniciarTraco(), solicitarSalvar()
Referências cruzadas	Histórias de Usuário: US 03, US 05
Pré-condições	Um Document ou uma nota deve estar aberto na interface de anotação.
Pós-condições	O traço manuscrito é exibido na tela. Quando o usuário aciona a ação de salvar, a anotação é enviada ao servidor e persistida. O servidor enfileira o processamento (OCR/HCR) da anotação.

Tabela 4. Contrato Anotação Manuscrita

2.4.3 DSS – Realizar Busca Global

Este diagrama, apresentado na Figura 4, modela o fluxo de interação para a história de usuário US 07, onde o usuário busca por um termo em toda a sua base de conhecimento. A Tabela 5 descreve o contrato do fluxo apresentado.

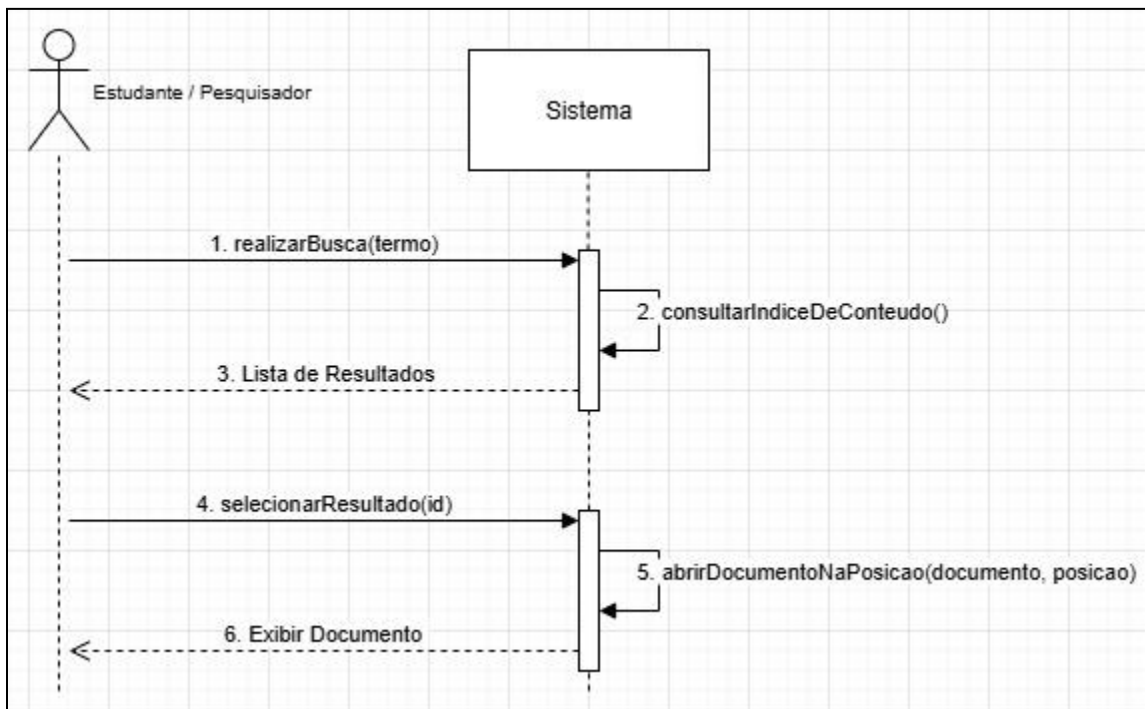


Figura 4. DSS para Realizar Busca Global

Contrato	Realizar Busca Global
Operação	realizarBusca(termo: string), selecionarResultado(id: string)
Referências cruzadas	História de Usuário: US 07
Pré-condições	O ator "Estudante / Pesquisador" deve estar autenticado no sistema
Pós-condições	Uma lista de resultados da busca é exibida para o usuário. O documento correspondente é aberto na localização exata do resultado, com o termo destacado.

Tabela 5. Contrato Realizar Busca Global

2.4.4 DSS – Explorar Conexão no Grafo

Este diagrama, apresentado na Figura 5, modela a jornada interativa de descoberta de conhecimento, que abrange as histórias de usuário US 08 e US 09. A primeira parte do fluxo, onde o usuário solicita e o sistema exibe o grafo, atende à necessidade de "ter uma visão geral" (US 08). A segunda

parte, que detalha a ação de clicar em uma conexão para ver o contexto e navegar até a fonte, representa a principal ação de exploração (US 09). O DSS, portanto, modela a experiência completa, desde a visualização macro até o "mergulho" em um *insight* específico. A Tabela 6 descreve o contrato do fluxo apresentado.

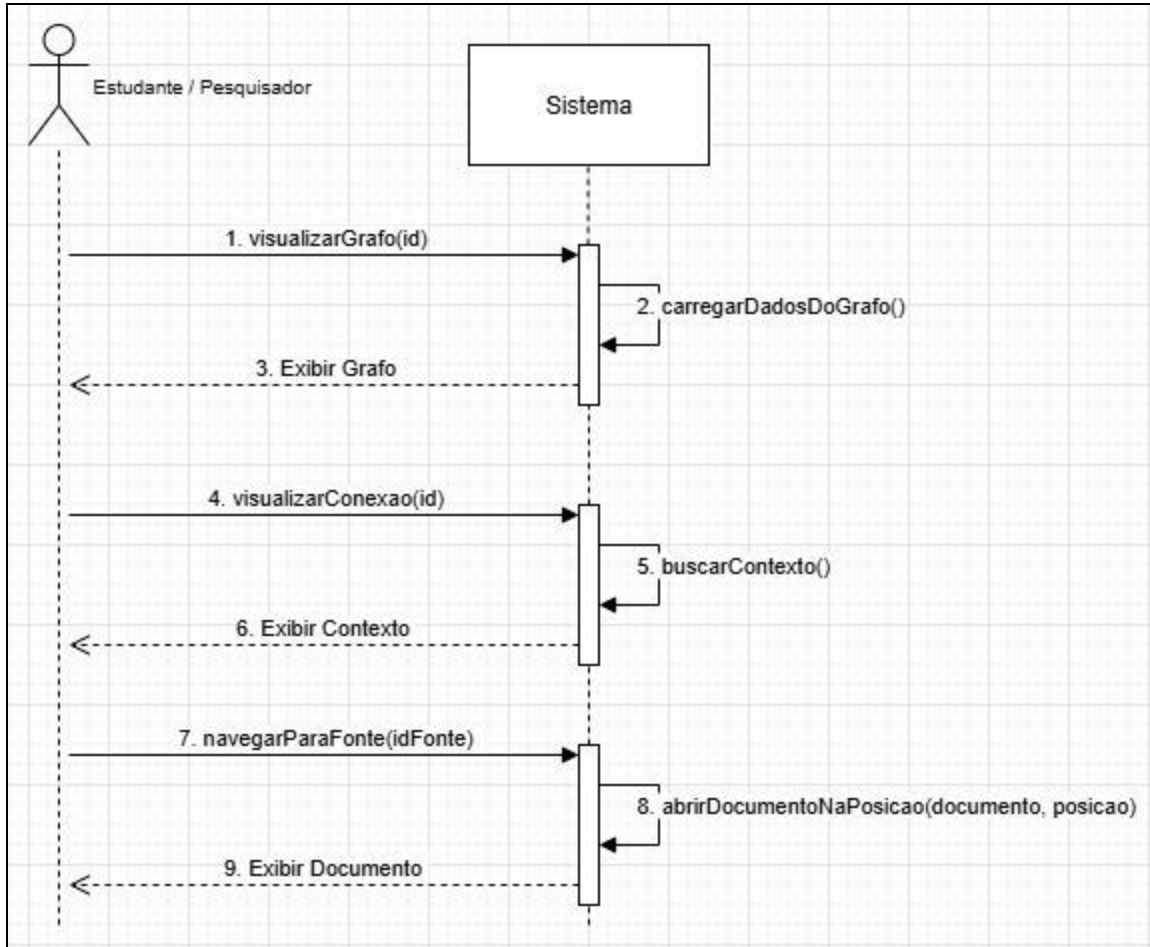


Figura 5. DSS para Explorar Conexão no Grafo

Contrato	Explorar Conexão no Grafo
Operação	visualizarGrafo(id)
Referências cruzadas	Histórias de Usuário: US 08, US 09
Pré-condições	O ator "Estudante / Pesquisador" deve estar autenticado no sistema
Pós-condições	O grafo de conhecimento do projeto é exibido de forma interativa na tela. O contexto da conexão selecionada (trechos de texto das fontes) é exibido para o usuário. O documento original é aberto na localização exata da fonte selecionada.

Tabela 6. Contrato Explorar Conexão no Grafo

3. Modelos de Projeto

Esta seção é dedicada à apresentação dos modelos de projeto que definem a arquitetura e o *design* detalhado do sistema Cognita. Serão apresentados os diagramas UML (*Unified Modeling Language*) que ilustram tanto a estrutura estática quanto o comportamento dinâmico do sistema. A modelagem começa com o Diagrama de Classes, que representa a espinha dorsal da arquitetura. Em seguida, serão detalhados os Diagramas de Sequência, Comunicação, Arquitetura, Estados, Componentes e Implantação utilizados para descrever a organização física e a distribuição do *software*.

3.1 Diagrama de Classes

O Diagrama de Classes, apresentado na Figura 6, é a principal representação da estrutura estática do sistema Cognita. Ele descreve as entidades fundamentais do domínio, seus atributos e operações, e os relacionamentos que existem entre elas, formando a base sobre a qual todas as funcionalidades serão construídas.

O modelo é centrado na classe User, que representa o ator do sistema. A partir dela, estabelece-se uma hierarquia de posse através de relações de composição (representadas pelo losango preto), que indicam um forte vínculo de ciclo de vida: se o "todo" é destruído, a "parte" também é. Assim, um User possui múltiplos Projects, que por sua vez compõem tanto os Documents (arquivos PDF) quanto as Annotations independentes. Esta modelagem permite que um projeto funcione como um "bloco de notas", contendo anotações que não estão atreladas a nenhum documento específico, atendendo à história de usuário US 04. Da mesma forma, um Document também compõe suas próprias Annotations, que representam os destaques e notas manuscritas feitas sobre o PDF.

A inovação central do sistema, o grafo de conhecimento, é representada por um conjunto de classes interligadas. A classe KnowledgeNode modela os conceitos abstratos extraídos pelo sistema (ex: "Semana de Arte Moderna"), funcionando como os nós do grafo. A relação de co-ocorrência entre dois KnowledgeNodes é descrita pela classe de associação KnowledgeEdge, cujo atributo *weight* indica em quantas sentenças eles co-ocorrem. A conexão crucial entre o conteúdo concreto e os conceitos abstratos é realizada pela classe Mention. Ela representa a ocorrência específica de um KnowledgeNode em uma localização exata e seu ciclo de vida depende inteiramente de sua fonte, sendo composta por um DocumentPage ou por uma Annotation. Esta classe é a ponte que permite a navegação contextual da visualização do grafo para a fonte original da informação, uma funcionalidade essencial do Cognita.

Para garantir a consistência e a segurança de tipo, o modelo também utiliza enumerações (<<enumeration>>) para atributos com um conjunto fixo de valores, como DocumentStatus, AnnotationStatus e NodeType.

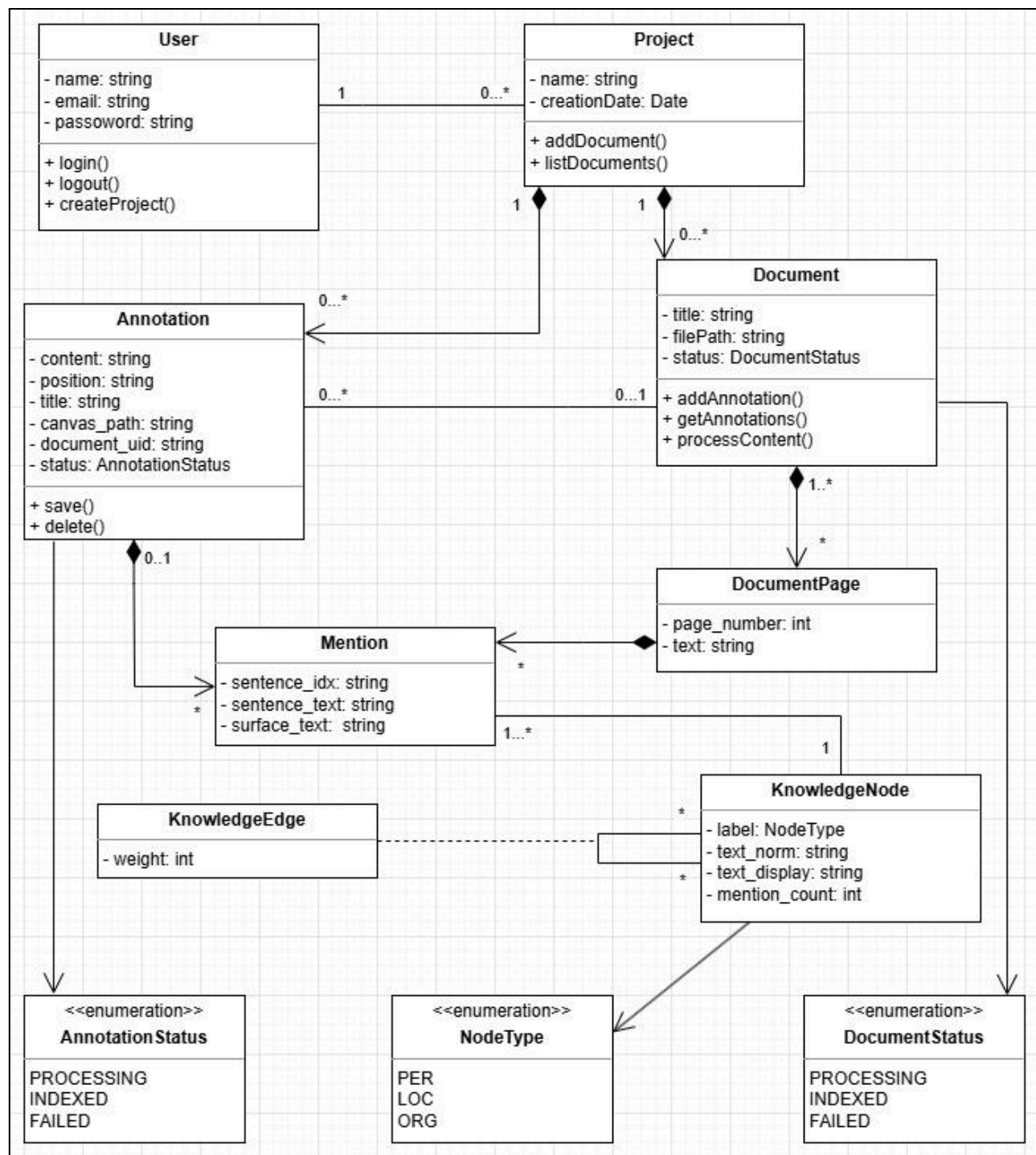


Figura 6. Diagrama de Classes do Sistema Cognita

3.2 Diagramas de Sequência

Os Diagramas de Sequência desta seção oferecem uma visão "caixa-branca" do sistema Cognita, detalhando como os objetos internos colaboram para realizar os principais casos de uso. Diferente dos Diagramas de Sequência do Sistema (DSS), que focam na fronteira do sistema, estes diagramas aprofundam a modelagem,

ilustrando a troca de mensagens entre os componentes da interface, os controladores, os serviços de *back-end* e as camadas de persistência de dados.

3.2.1 Diagrama de Sequência – Fazer Anotação Manuscrita

A Figura 7 detalha a interação para as histórias US 03 e US 05. A interação começa com o ator desenhando na Interface, que repassa os eventos para o :ControllerAnotacao. Dentro do fragmento de *loop*, o controlador comanda a renderização do traço localmente no *canvas*, garantindo *feedback* instantâneo à caneta. Ao acionar a ação de salvar, o ator dispara solicitarSalvar(); o :ControllerAnotacao então envia a anotação ao :APIServidor, que a persiste e enfileira seu processamento (OCR/HCR) de forma assíncrona.

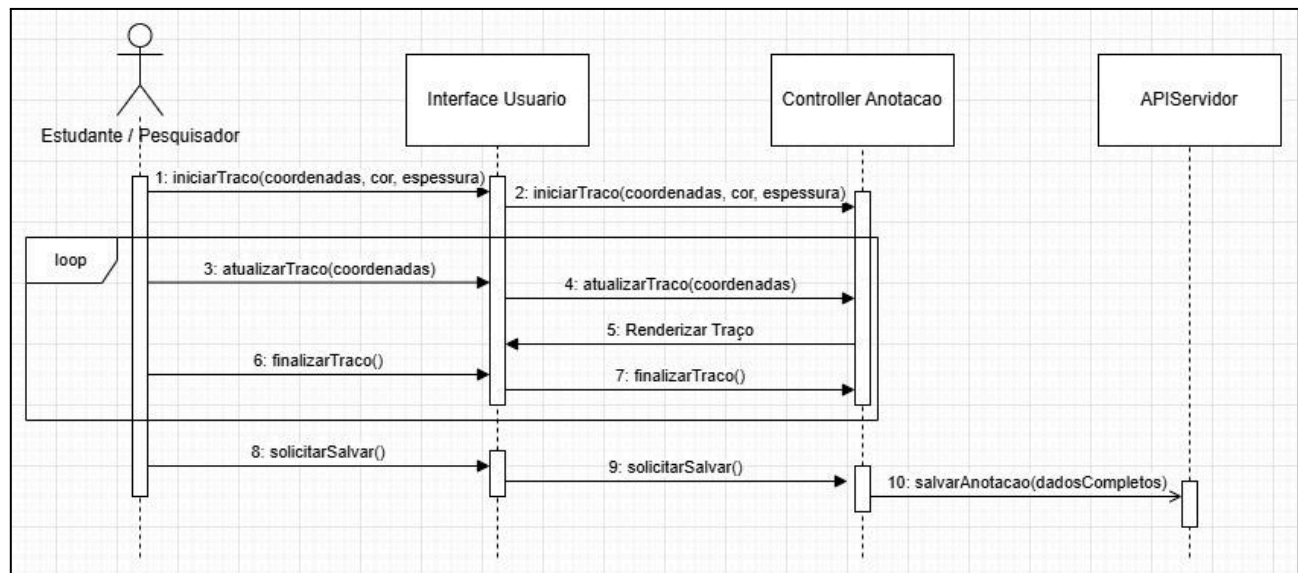


Figura 7. Diagrama de Sequência - Fazer Anotação Manuscrita

3.2.2 Diagrama de Sequência - Realizar Upload de Documento

A Figura 8 modela o fluxo para as histórias de usuário US 01 e US 06. Este diagrama evidencia a arquitetura de processamento assíncrono do sistema. Após o usuário iniciar o *upload*, o :ControllerBiblioteca envia os arquivos para o :APIServidor. O servidor, por sua vez, cria um registro inicial do documento no banco de dados com o estado "PROCESSANDO" e imediatamente enfileira a tarefa de processamento pesado (OCR/HCR) para o :ProcessadorConteudo. Uma confirmação é retornada ao usuário sem esperar o término do processamento, garantindo que a aplicação não trave durante operações demoradas.

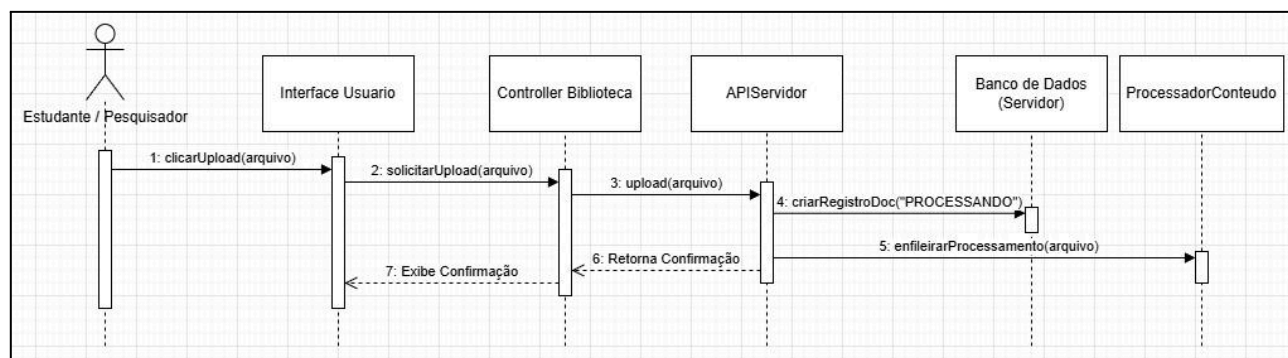


Figura 8. Diagrama de Sequência - Realizar Upload de Documento

3.2.3 Diagrama de Sequência – Realizar Busca Global

A Figura 9 ilustra a jornada completa do usuário para a história US 07. O fluxo é dividido em duas fases. Na primeira, o usuário digita um termo, que é enviado através das camadas de controle até o :APIServidor. O servidor consulta um índice de conteúdo otimizado no banco de dados para retornar uma lista de resultados. Na segunda fase, ao selecionar um resultado, o controlador comanda a interface para abrir o documento na posição da ocorrência, completando o ciclo de busca e recuperação da informação.

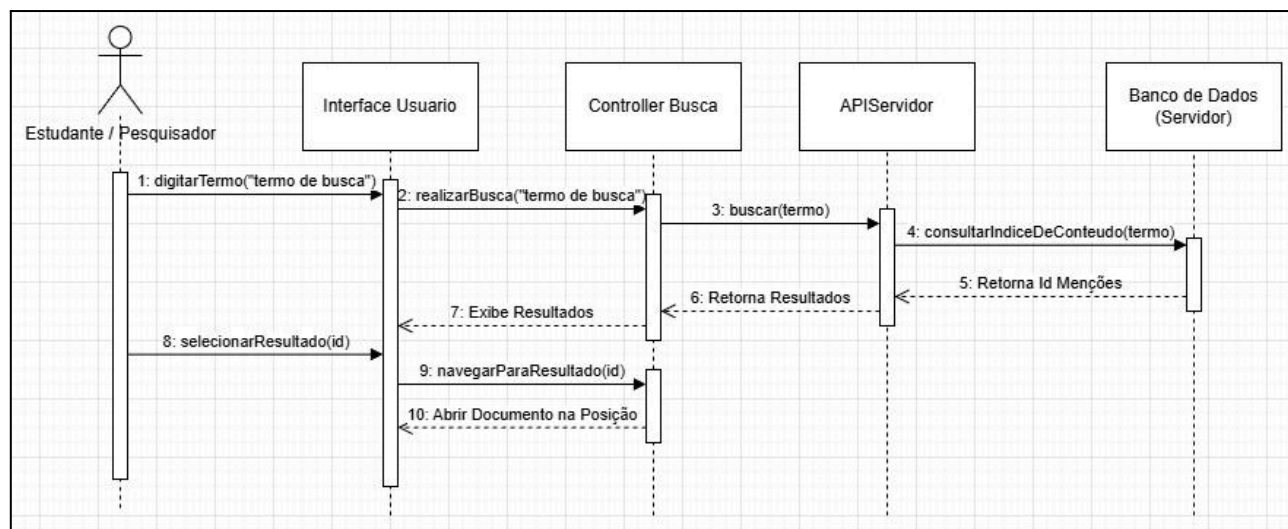


Figura 9. Diagrama de Sequência - Realizar Busca Global

3.2.4 Diagrama de Sequência – Explorar Conexão no Grafo

A Figura 10 detalha a interação com a funcionalidade mais inovadora do Cognita, correspondente às histórias US 08 e US 09. O diagrama modela a jornada de descoberta em três fases: primeiro, o usuário solicita a visualização do grafo de um projeto, e o sistema busca os nós e arestas no banco de dados para exibi-lo. Segundo, ao clicar em uma conexão, o sistema busca as menções associadas para exibir o painel de contexto. Terceiro, ao selecionar uma fonte no painel de contexto, o sistema navega o usuário diretamente para a localização daquela informação no documento original, cumprindo a principal proposta de valor do projeto.

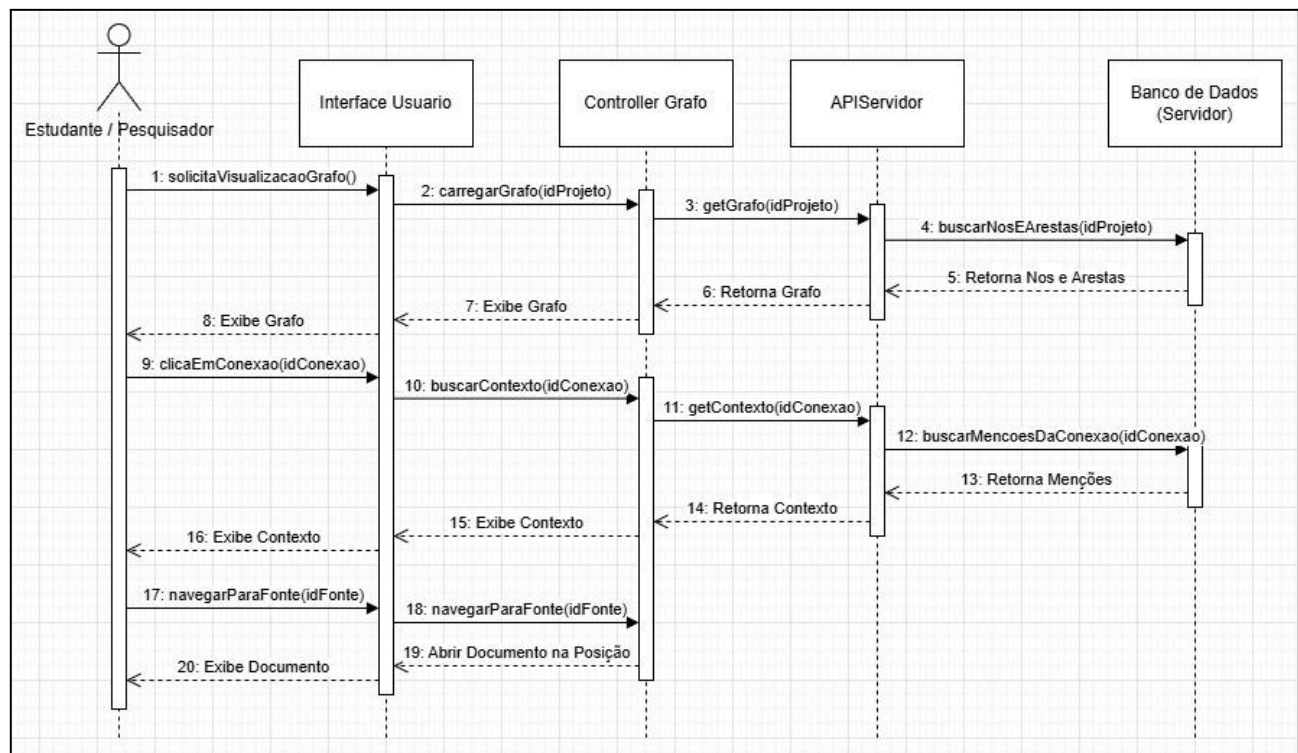


Figura 10. Diagrama de Sequência – Explorar Conexão no Grafo

3.3 Diagramas de Comunicação

Os Diagramas de Comunicação oferecem uma visão alternativa das interações modeladas nos Diagramas de Sequência. Enquanto os diagramas de sequência enfatizam a ordem temporal das mensagens, os diagramas de comunicação focam na estrutura das interações e nos relacionamentos entre os objetos participantes.

Eles utilizam a mesma base de informações dos Diagramas de Sequência, mas representam o fluxo através de uma numeração sequencial das mensagens trocadas ao longo dos links que conectam os objetos. A seguir, são apresentados os diagramas de comunicação para os principais casos de uso do sistema Cognita.

3.3.1 Diagrama de Comunicação – Fazer Anotação Manuscrita

A Figura 11 ilustra a colaboração entre os objetos para as histórias US 03 e US 05. O diagrama destaca os links necessários para criar uma anotação manuscrita, desde a captura do gesto na interface e a renderização local do traço até o envio explícito ao *backend* pelo usuário, via API REST, quando ele aciona a ação de salvar.

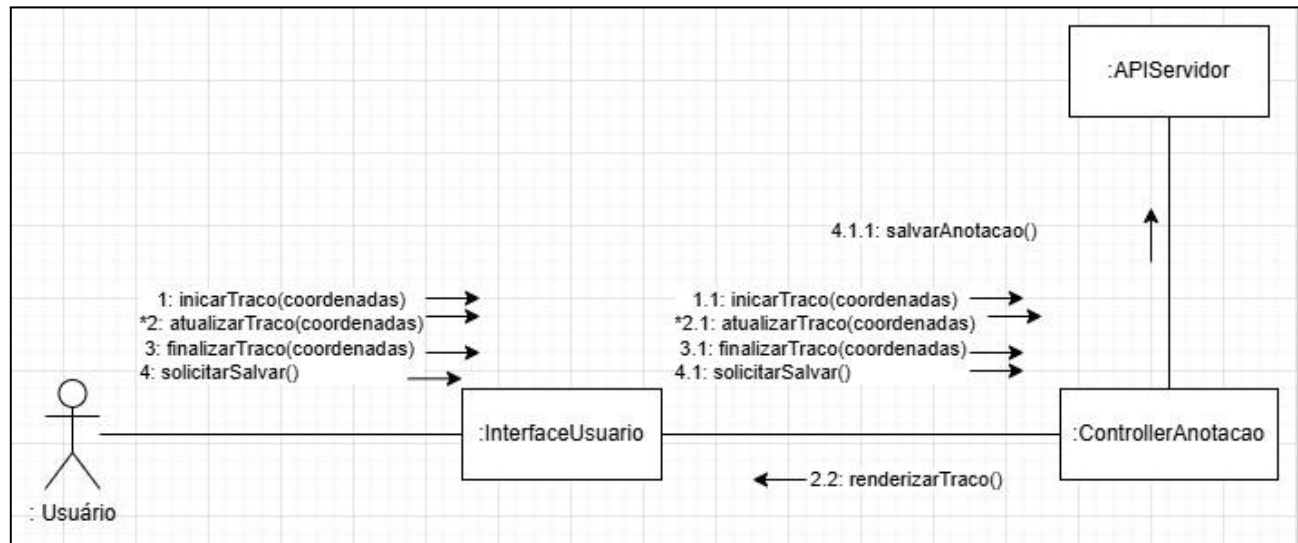


Figura 11. Diagrama de Comunicação – Fazer Anotação Manuscrita

3.3.2 Diagrama de Comunicação – Realizar Upload de Documento

A Figura 12 representa a colaboração de objetos para as histórias de usuário US 01 e US 06. O diagrama mostra como o cliente (:ControllerBiblioteca) se comunica apenas com a porta de entrada do servidor (:APIServidor), que por sua vez orquestra as ações internas de salvar o registro no banco de dados e enfileirar a tarefa de processamento para o :ProcessadorConteudo de forma assíncrona.

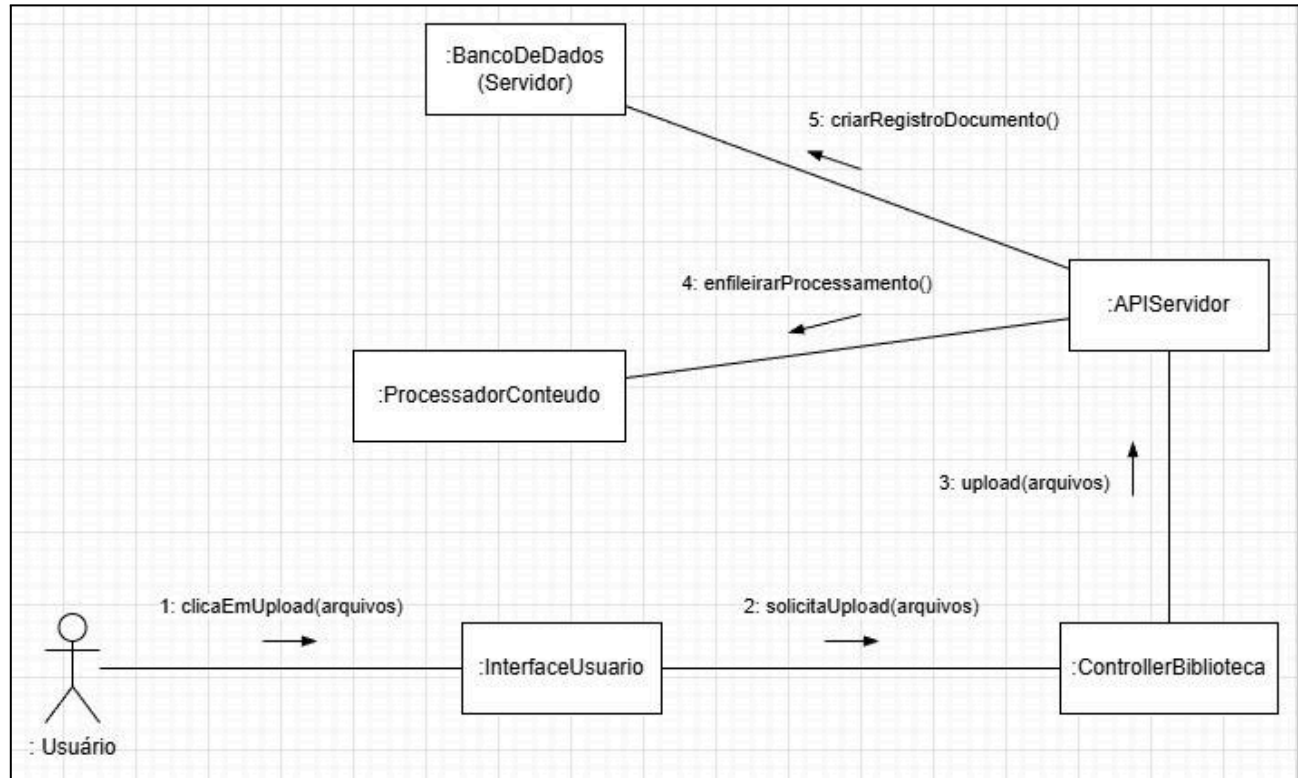


Figura 12. Diagrama de Comunicação – Realizar Upload de Documento

3.3.3 Diagrama de Comunicação – Realizar Busca Global

A Figura 13 detalha a colaboração para a história de usuário US 07. O diagrama foca nas relações entre os componentes para realizar a busca completa. Ele mostra a cadeia de comunicação, desde a consulta do usuário na interface até a chamada à API, que consulta o índice de conteúdo no banco de dados e, em uma segunda fase, como a seleção de um resultado na interface aciona o controlador para exibir o documento na posição correta.

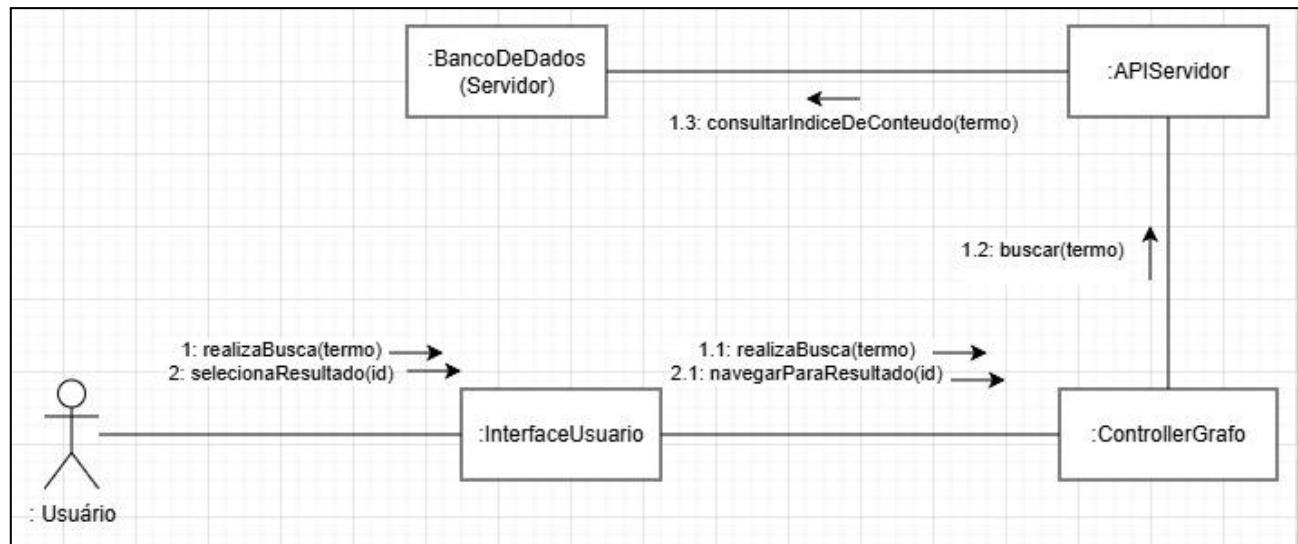


Figura 13. Diagrama de Comunicação – Realizar Busca Global

3.3.4 Diagrama de Comunicação – Explorar Conexão no Grafo

A Figura 14 ilustra a colaboração de objetos para as histórias de usuário US 08 e US 09, que representam a principal funcionalidade do sistema. O diagrama mostra as relações necessárias para a jornada completa do usuário: primeiro, a busca dos dados do grafo; segundo, a busca pelo contexto de uma conexão específica; e terceiro, a navegação para a fonte original no documento, demonstrando a complexa orquestração entre os componentes para entregar o *insight* ao usuário.

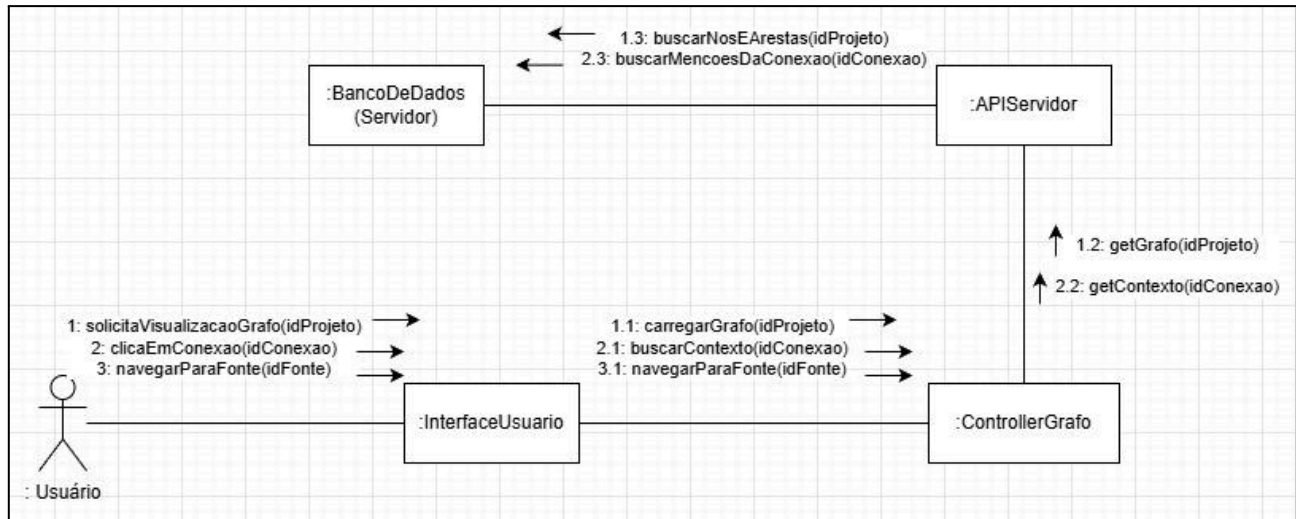


Figura 14. Diagrama de Comunicação – Explorar Conexão no Grafo

3.4 Arquitetura

A arquitetura do sistema Cognita foi projetada para atender a requisitos de performance, escalabilidade e confiabilidade, garantindo uma experiência de usuário fluida e responsiva, especialmente nas funcionalidades de anotação. O modelo arquitetural escolhido é uma Arquitetura Cliente-Servidor em Camadas, na qual a aplicação cliente concentra a interação e a renderização da interface, enquanto o servidor centraliza a persistência dos dados e o processamento pesado.

A arquitetura é dividida em dois macro componentes principais: a Aplicação Cliente, que roda no *tablet* do usuário, e o Servidor *Backend*, que centraliza a lógica de negócio pesada e a persistência de dados.

O cliente, desenvolvido em React Native (Expo) e voltado para *tablets*, é o coração da interação do usuário: concentra a biblioteca de documentos, o leitor de PDF, o canvas de anotação e a visualização do grafo de conhecimento. Ele se comunica com o *backend* exclusivamente por meio de uma API REST, que é responsável por toda a persistência dos dados e pelo processamento pesado. Para garantir uma experiência fluida na anotação manuscrita, o desenho é renderizado localmente no *canvas*, oferecendo *feedback* imediato à caneta. Quando o usuário aciona a ação de salvar, a anotação é enviada ao servidor para ser persistida e processada (OCR/HCR).

O cliente é estruturado em duas camadas lógicas:

- Camada de Apresentação (UI): responsável por toda a renderização da interface, incluindo o *canvas* de anotação (renderizado em WebView) e a visualização do grafo de conhecimento. Captura os gestos do usuário e exibe os dados retornados pelo servidor.

- Camada de Controle (Lógica da Aplicação): onde residem os controladores (ControllerAnotacao, ControllerBiblioteca, etc.). Orquestra o fluxo de dados, gerencia o estado da aplicação (autenticação, projeto e documento selecionados) e se comunica com o *backend* através da API REST.

O servidor, *backend*, é responsável pelas operações que exigem processamento intensivo, persistência de dados centralizada e segurança. Ele também segue uma arquitetura em camadas para garantir a separação de responsabilidades e a manutenibilidade.

- *Gateway* de API (APIServidor): É a única porta de entrada para o *backend*. Ele expõe uma API REST para o cliente, validando todas as requisições e orquestrando as chamadas para os serviços internos.
- Camada de Serviço: Contém a lógica de negócio principal. É aqui que as requisições são processadas. Uma decisão arquitetural chave foi separar os serviços síncronos (como busca de dados) dos assíncronos.
- Processador de Conteúdo Assíncrono (ProcessadorConteudo): Um serviço especializado e desacoplado que opera em uma fila de tarefas. Ele é responsável por todas as operações pesadas e demoradas, como o processamento de OCR/HCR e a análise de PLN para a construção do grafo de conhecimento. Esta separação garante que a API principal permaneça sempre responsiva.
- Camada de Dados: Gerencia a comunicação com o banco de dados centralizado na nuvem, onde todos os dados dos usuários são armazenados de forma segura e persistente.

Como vantagens desta escolha de arquitetura:

- Responsividade na anotação: o traço da caneta é renderizado localmente no *canvas*, oferecendo *feedback* visual imediato; o processamento pesado (OCR/HCR e construção do grafo) ocorre de forma assíncrona no servidor, sem travar a interface.
- Consistência e centralização dos dados: toda a base de conhecimento é persistida de forma centralizada no servidor, garantindo uma única fonte da verdade e facilitando *backup* e um futuro acesso a partir de múltiplos dispositivos.
- Caminho aberto para escalabilidade: o ProcessadorConteudo é executado em segundo plano (via BackgroundTasks do FastAPI), garantindo que requisições à API não fiquem bloqueadas por operações pesadas como OCR/HCR. Embora hoje ele rode no mesmo processo da API, a separação lógica entre os dois componentes deixa a porta aberta para promovê-lo a um *worker* independente (com fila/processos dedicados) quando o volume justificar, sem impactar o restante da arquitetura.
- Manutenibilidade: a divisão clara em camadas e componentes, tanto no cliente quanto no servidor, facilita a manutenção, os testes e a evolução do sistema.

A Figura 15 ilustra a arquitetura do Cognita dividida em dois pacotes principais: Cliente e Servidor. O pacote Cliente contém os sub-pacotes que representam suas camadas lógicas (UI, Controladores). O pacote Servidor contém os sub-pacotes que representam seus componentes internos (API, Serviços, Processadores, Dados). A seta de dependência tracejada mostra a única via de comunicação permitida: o pacote Cliente depende exclusivamente do pacote API do servidor, que atua como um *gateway*, encapsulando toda a complexidade do *backend*.

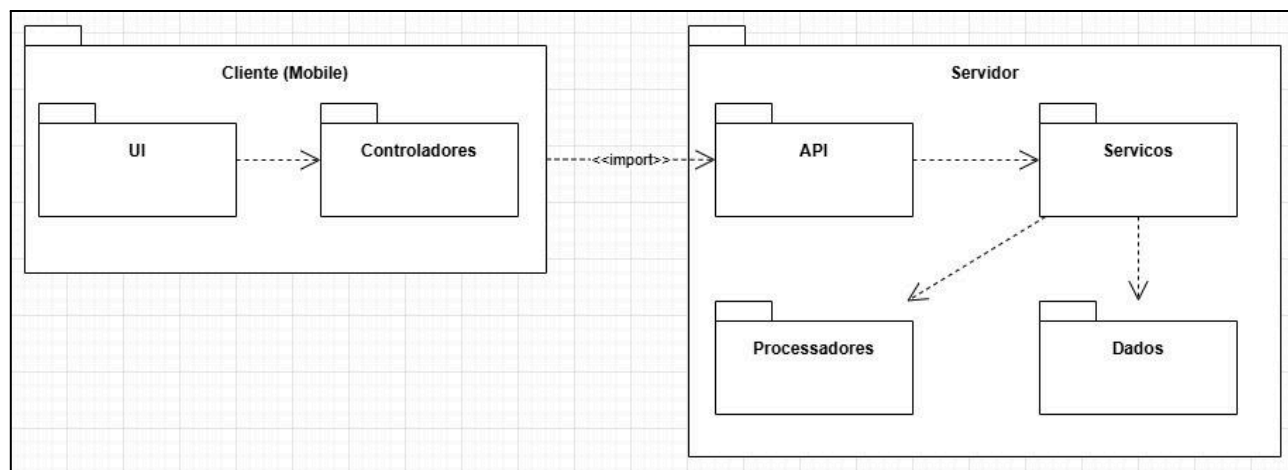


Figura 15. Diagrama de Pacotes

3.5 Diagramas de Estados

Para o sistema Cognita, o objeto mais relevante para este tipo de modelagem é a classe Document, devido ao seu ciclo de vida que envolve o processamento assíncrono de conteúdo. A Figura 16 ilustra a máquina de estados de um Document, desde o *upload* até a indexação do seu texto.

Após ser criado, um Document entra no estado Aguardando Processamento. Quando o serviço de *backend* inicia a tarefa, o documento transita para Processando Conteúdo. Este estado possui atividades internas (do /) que representam a extração de texto via OCR e HCR. Se a extração for concluída com êxito, o evento *processamentoConcluidoComSucesso* leva o documento ao estado final Indexado; ao entrar (entry /) nesse estado, seu *status* é atualizado e a interface é notificada, tornando o documento pesquisável. Caso ocorra qualquer erro, o evento *erroNoProcessamento* leva o documento ao estado Falha no Processamento, onde são executadas ações de registro de erro.

Vale notar que a construção do grafo de conhecimento não faz parte deste ciclo de vida: é um processo separado, executado em nível de projeto sobre os documentos e anotações já indexadas.

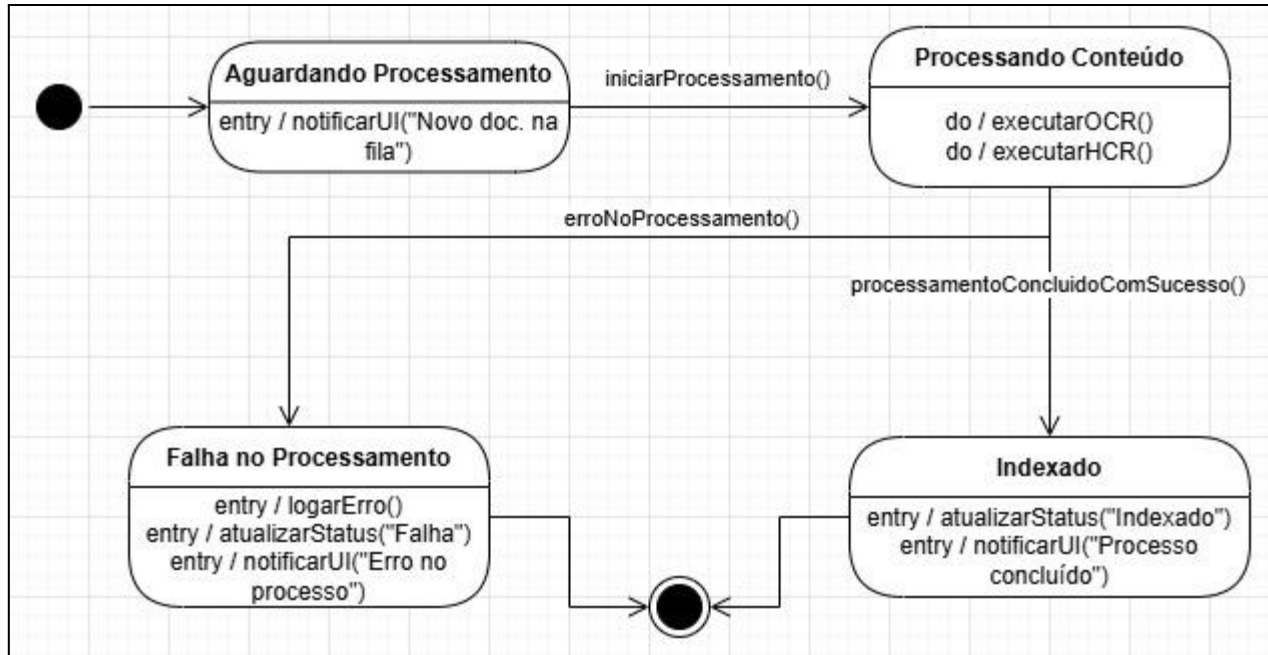


Figura 16. Diagrama de Estados

3.6 Diagrama de Componentes e Implantação

Esta seção finaliza a modelagem de projeto, focando na estrutura física e na distribuição dos artefatos de *software*. O Diagrama de Componentes, em particular, move a representação da arquitetura de uma visão puramente lógica (como no Diagrama de Pacotes) para uma visão mais concreta, que descreve os componentes de *software* reais, suas interfaces e as dependências entre eles

3.6.1 Diagrama de Componentes

O Diagrama de Componentes, apresentado na Figura 17, ilustra a arquitetura física do sistema Cognita. Ele detalha os principais componentes de *software* que constituem a aplicação cliente e o servidor *backend*, bem como as tecnologias e serviços de terceiros dos quais o sistema depende para operar.

O diagrama é dividido em dois grandes blocos: o Cliente e o Servidor.

É importante notar que a escolha de componentes externos, como o Serviço de Armazenamento de Objetos e a API de Visão, é guiada pela disponibilidade de um plano *free-tier* que atende às restrições de custo do projeto. Ciente de que estes planos estão sujeitos a limites e mudanças, a arquitetura do *backend* foi projetada para ser tecnologicamente agnóstica. Este *design* garante que qualquer um desses componentes possa ser substituído por outra alternativa, com impacto mínimo na lógica de negócio central do sistema.

No lado do Cliente, temos o componente principal Aplicação Cliente, que será implementado em React Native. Este componente expõe a Interface de Usuário (UI) para o ator.

No lado do Servidor, a arquitetura é composta por um componente central, o Servidor Backend (API), que expõe uma REST API como sua interface principal. Este componente é o único ponto de contato para o cliente, encapsulando toda a lógica de negócio e as interações com os demais serviços. Para executar suas funções, o Servidor Backend depende de um conjunto de componentes especializados:

- **Armazenamento de Objetos:** Componente responsável por armazenar de forma segura e escalável os artefatos binários do sistema: arquivos PDF enviados pelos usuários e os dados de canvases das anotações. A opção por um serviço dedicado, em vez de manter esses binários no próprio banco de grafo, é a prática adequada para arquivos desse porte e permite o acesso controlado por meio de URLs assinadas. Por seguir o padrão S3, o componente pode ser substituído por outro provedor sem impacto na lógica de aplicação, reforçando o princípio de arquitetura agnóstica a fornecedor adotado pelo *backend*.
- **API de Visão:** Um Serviço Externo do qual o *backend* depende para realizar as tarefas de Reconhecimento de Escrita à Mão (HCR) e Reconhecimento Óptico de Caracteres (OCR).
- **Banco de Dados de Grafo:** Componente central de persistência do servidor, responsável por armazenar todos os dados estruturados do sistema, incluindo usuários, projetos e, crucialmente, os nós e arestas do grafo de conhecimento.
- **Modelos de PLN:** Representa a dependência do *backend* com o *pipeline* de Processamento de Linguagem Natural responsável pela construção do grafo de conhecimento. A etapa de Reconhecimento de Entidades Nomeadas (NER) utiliza o modelo pré-treinado spaCy pt_core_news_lg, otimizado para o português, que identifica entidades dos tipos Pessoa (PER), Local (LOC) e Organização (ORG). Já a Extração de Relações (RE) não emprega um modelo dedicado: as relações entre entidades são inferidas por co-ocorrência: duas entidades que aparecem na mesma sentença são conectadas por uma aresta, cujo peso corresponde ao número de sentenças em que co-ocorrem. Por ser uma abordagem determinística, dispensa modelos generativos (LLMs) e elimina o risco de relações inventadas.

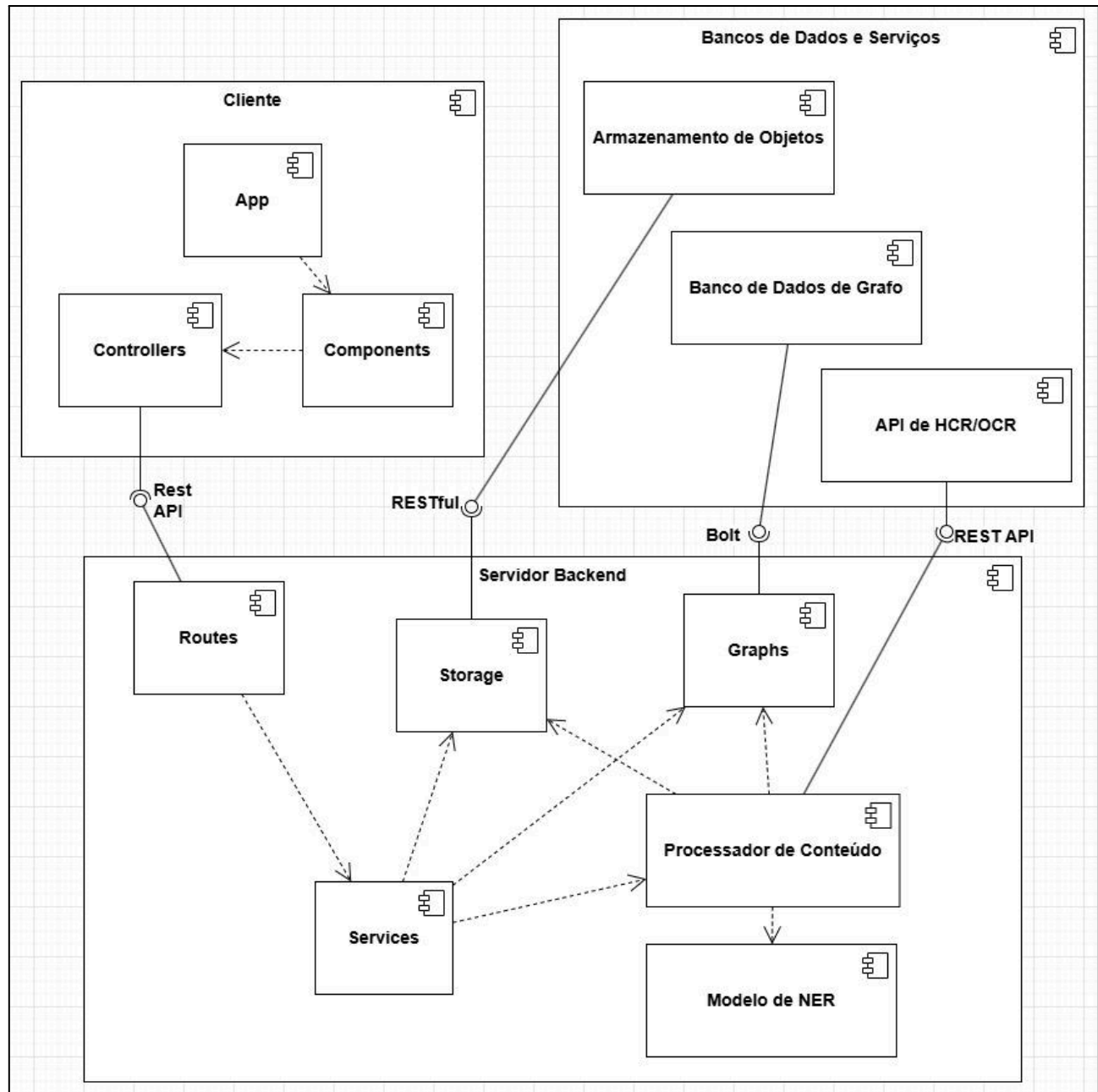


Figura 17. Diagrama de Componentes

3.6.2 Diagrama de Implantação

O Diagrama de Implantação complementa o Diagrama de Componentes ao modelar a topologia física do sistema. Ele ilustra como os artefatos de *software* (os componentes executáveis) são alocados nos nós de *hardware* ou ambientes de execução, e como esses nós se comunicam entre si.

A Figura 18 apresenta a arquitetura de implantação do sistema Cognita. Uma decisão de design importante neste diagrama é a representação dos serviços de nuvem de forma abstrata, focando em seu papel arquitetural

em vez de um provedor específico. Isso reforça a independência tecnológica do sistema, onde a implementação concreta é um detalhe que pode ser alterado sem impactar a arquitetura de implantação geral.

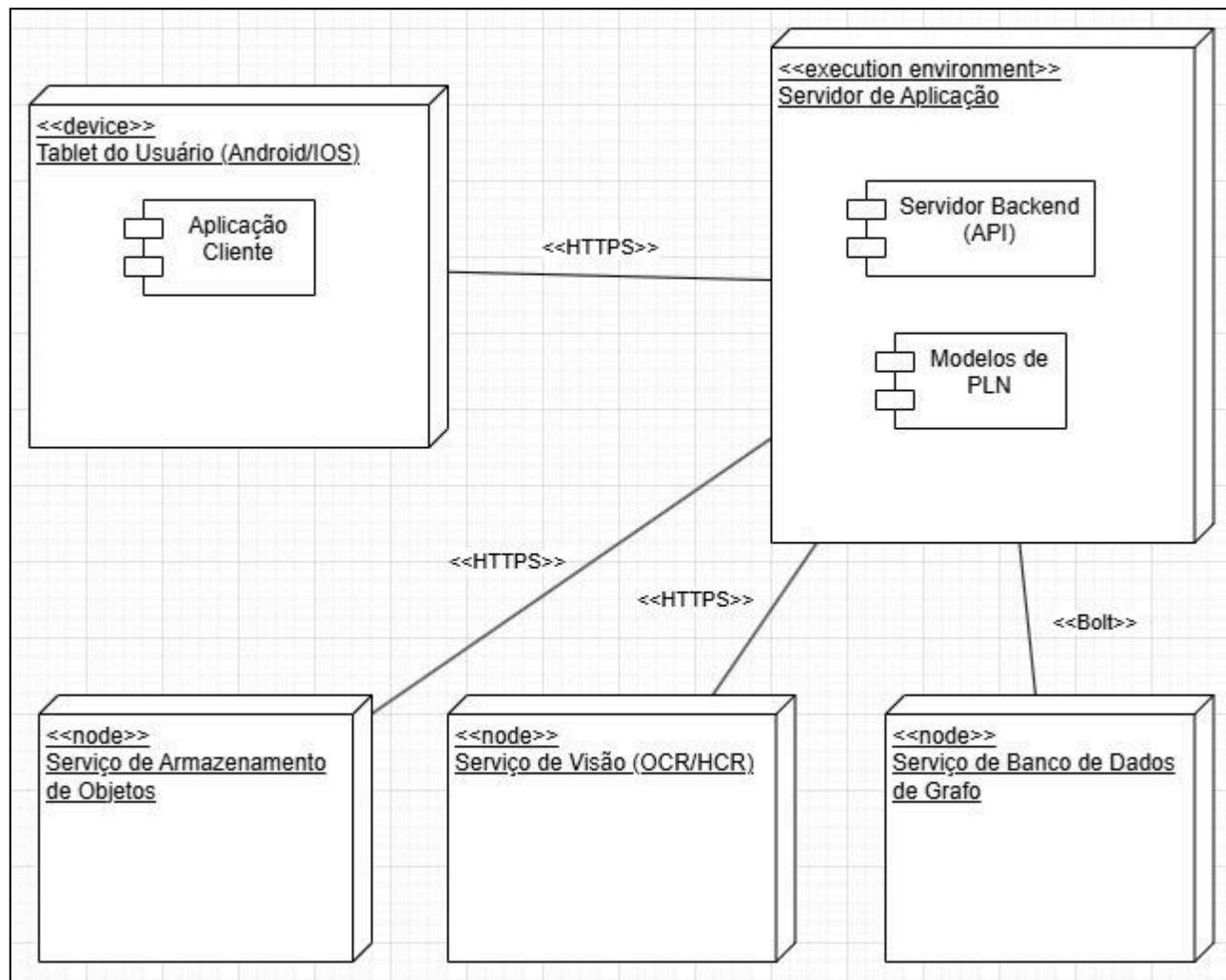


Figura 18. Diagrama de Implantação

4. Projeto de Interface com Usuário

Esta seção apresenta os wireframes de baixa fidelidade que esboçam a estrutura visual e o fluxo de navegação do sistema Cognita. O objetivo destes esboços é ilustrar as principais telas com as quais o ator "Estudante / Pesquisador" irá interagir, demonstrando como as funcionalidades descritas nos casos de uso e histórias de usuário se manifestam na interface. O *design* foi concebido com foco em uma experiência minimalista e funcional, otimizada para a interação em *tablets*.

4.1 Esboço das Interfaces Comuns a Todos os Atores

As interfaces a seguir representam o fluxo principal do usuário, desde a autenticação até a interação com as funcionalidades centrais de anotação e visualização do grafo de conhecimento.

As telas de autenticação, mostradas na Figura 19 e Figura 20, foram projetadas para serem simples e diretas, garantindo um acesso seguro e rápido à plataforma.

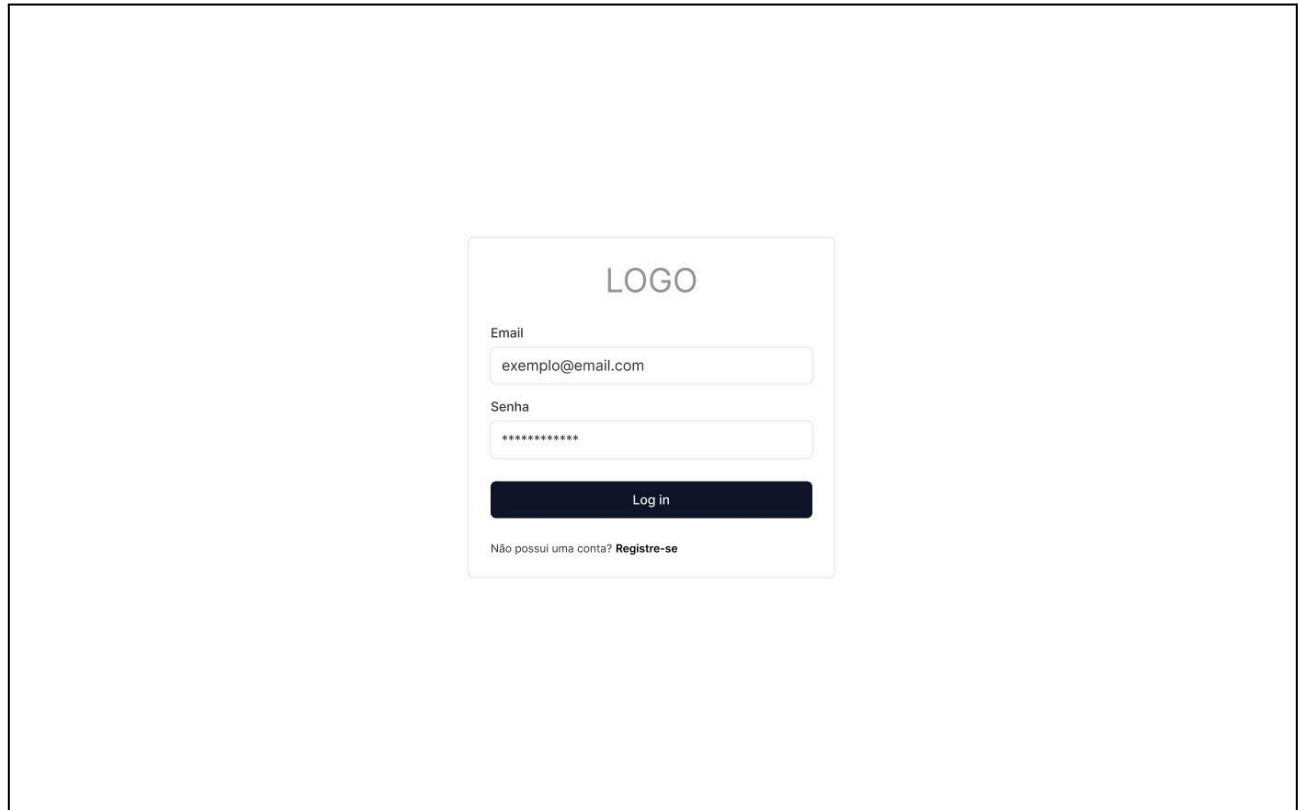
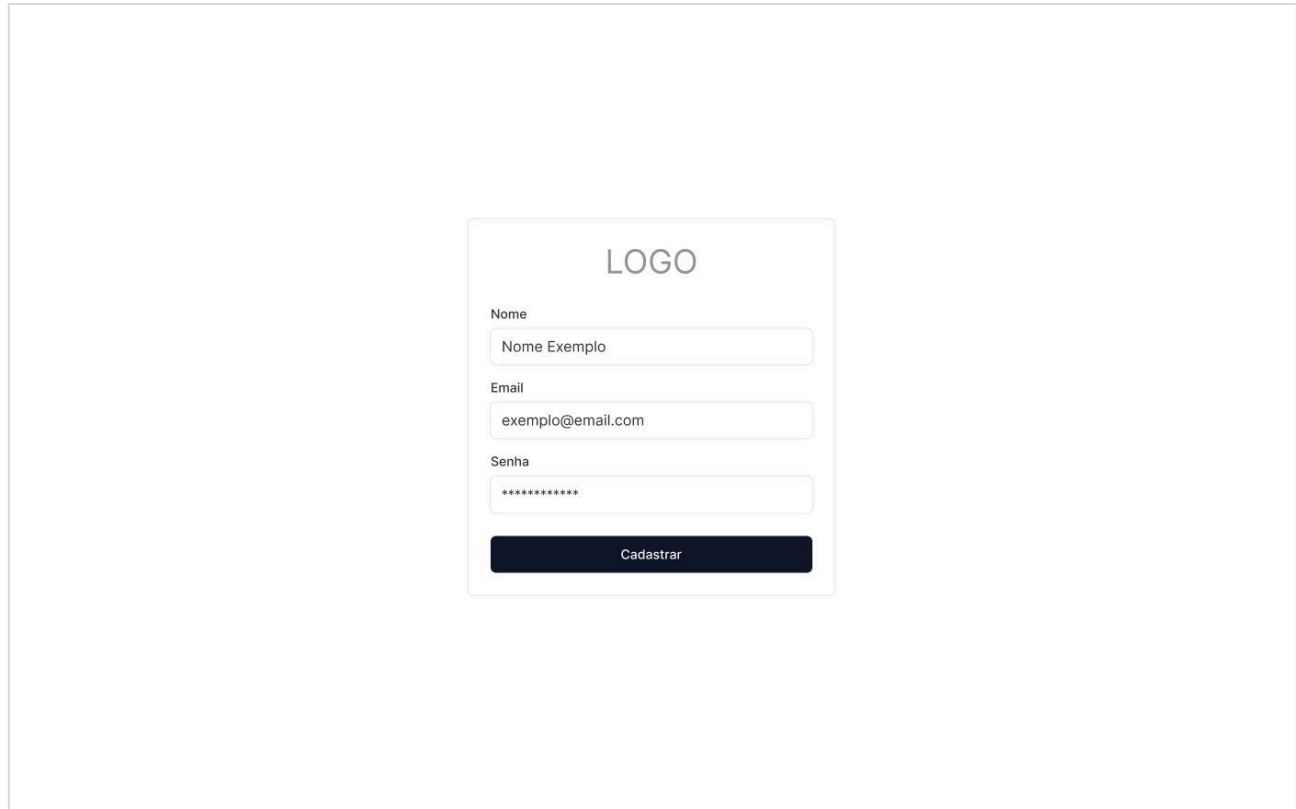


Figura 19. Tela de Login

A Figura 19 ilustra a tela de *login*, o ponto de entrada para usuários já cadastrados no sistema. A interface solicita apenas as credenciais essenciais (email e senha) para manter o foco na ação de acesso.



A imagem mostra a tela de cadastro de um sistema. No topo, há um espaço reservado para o "LOGO". Abaixo, há três campos de entrada de texto: "Nome" com o valor "Nome Exemplo", "Email" com o valor "exemplo@email.com", e "Senha" com caracteres ocultos por pontos. Um botão escuro com o texto "Cadastrar" está posicionado na base do formulário.

Figura 20. Tela de Cadastro

A Figura 20 apresenta a tela de cadastro para novos usuários. O formulário solicita as informações mínimas necessárias para a criação de uma conta: nome, email e senha.

Uma vez autenticado, o usuário tem acesso à área principal do sistema. A interface é organizada em um leiaute de múltiplos painéis para facilitar a navegação e a interação com os conteúdos, como demonstrado nas figuras subsequentes.

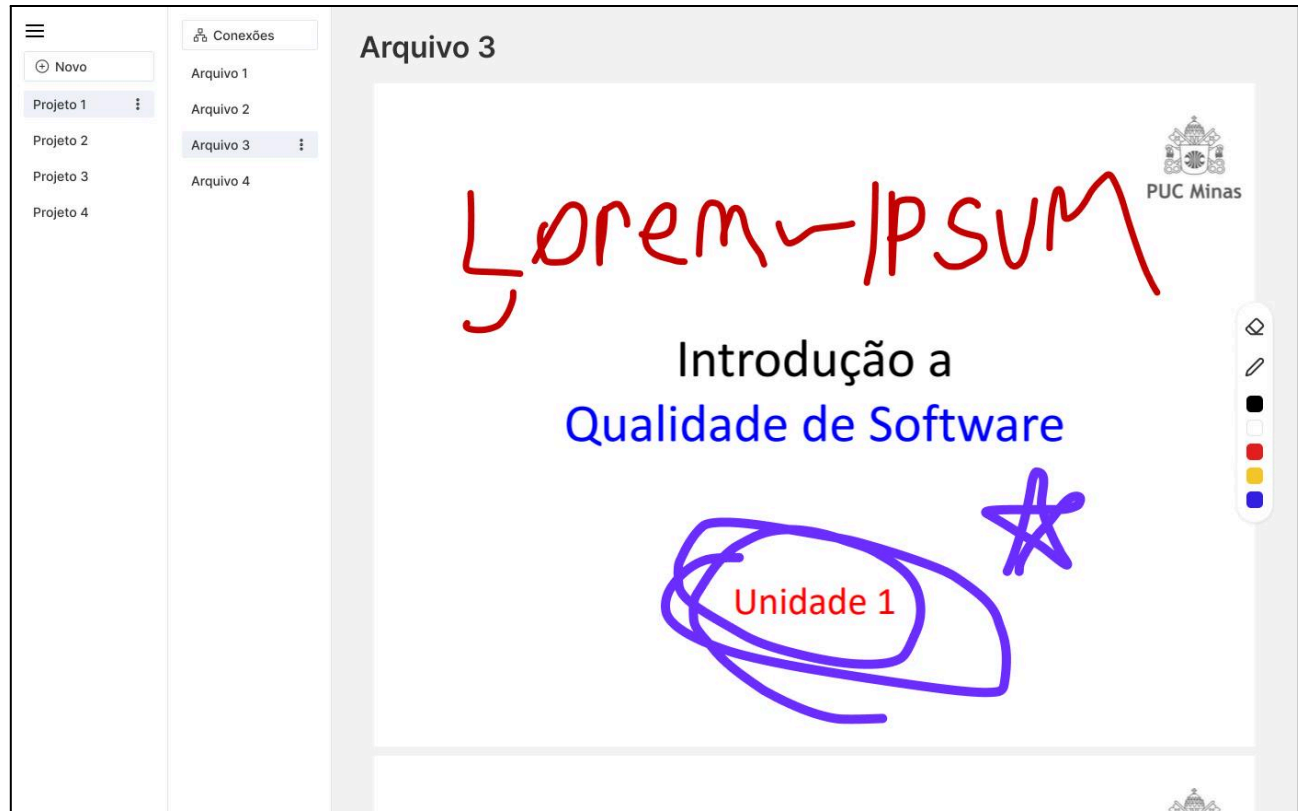


Figura 21. Tela de anotação em pdf

A Figura 21 exibe uma das telas centrais do Cognita: o leitor de PDF com funcionalidades de anotação. O painel mais à esquerda permite a navegação entre os diferentes "Projetos" do usuário. O segundo painel lista os arquivos contidos no projeto selecionado. A área principal, à direita, é dedicada à visualização do documento, onde o usuário pode realizar anotações manuscritas diretamente sobre o conteúdo com uma caneta *stylus*, como exemplificado pelos traços e destaques na imagem.

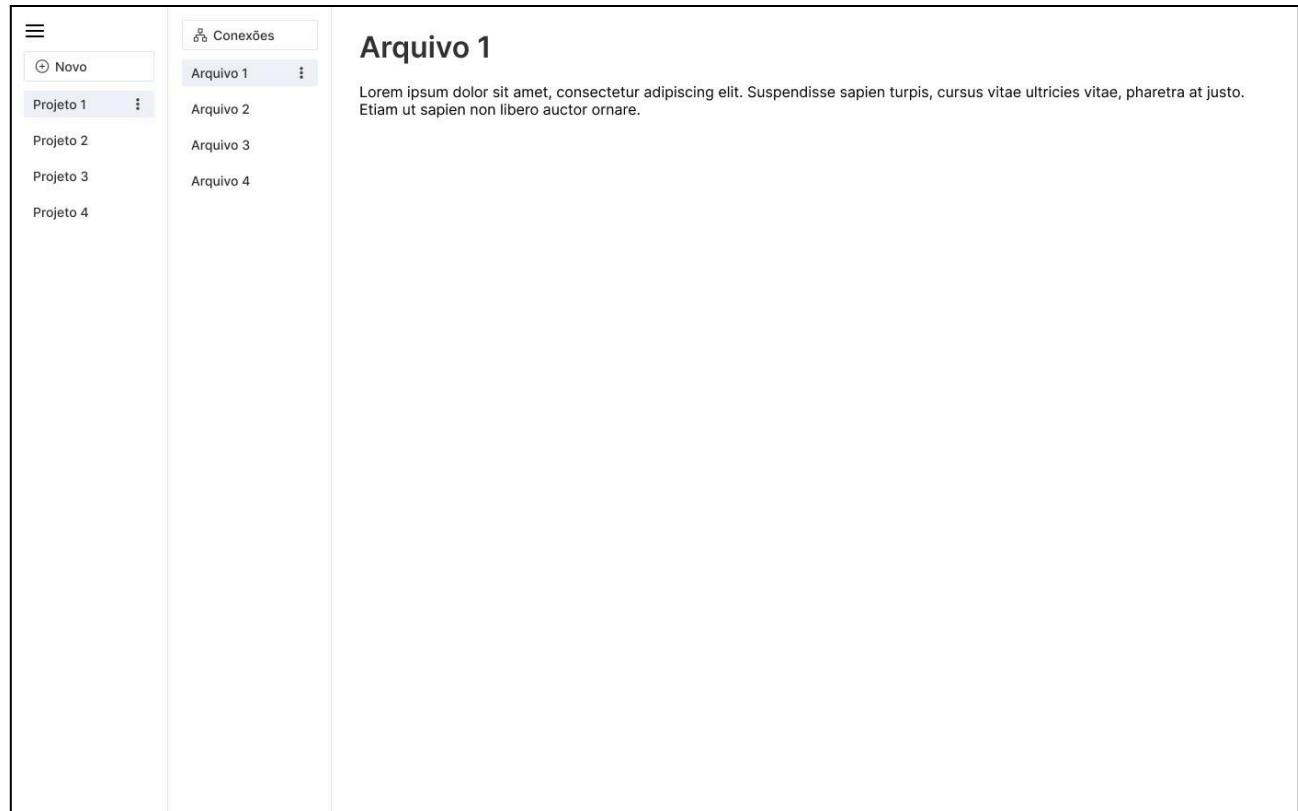


Figura 22. Tela de anotação digitada

A Figura 22 mostra a interface para uma nota simples contendo texto digitado. Esta tela demonstra a capacidade do sistema de suportar anotações textuais tradicionais.

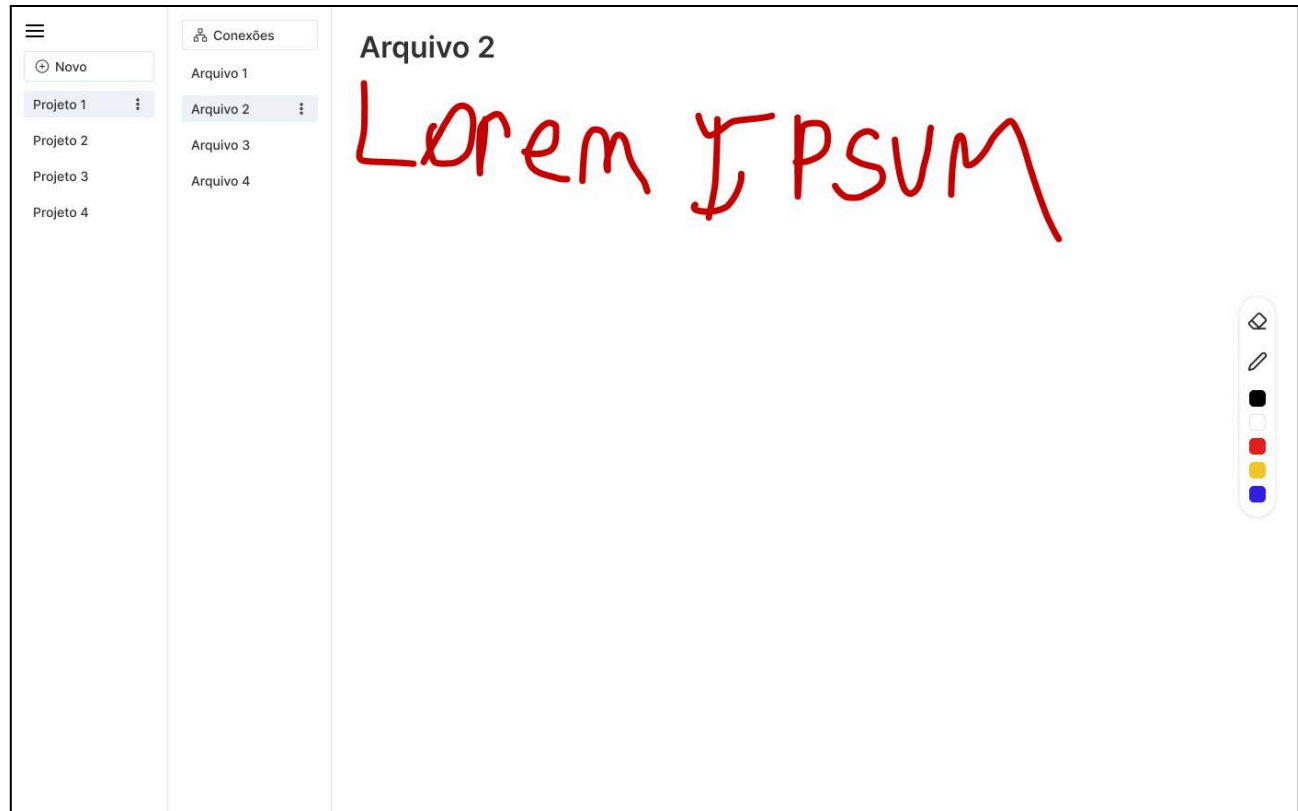


Figura 23. Tela de anotação manuscrita

Complementando a funcionalidade anterior, a Figura 23 ilustra a criação de uma nota com conteúdo puramente manuscrito. Esta funcionalidade é essencial para a proposta de valor do sistema, permitindo que o usuário capture ideias de forma livre e natural.

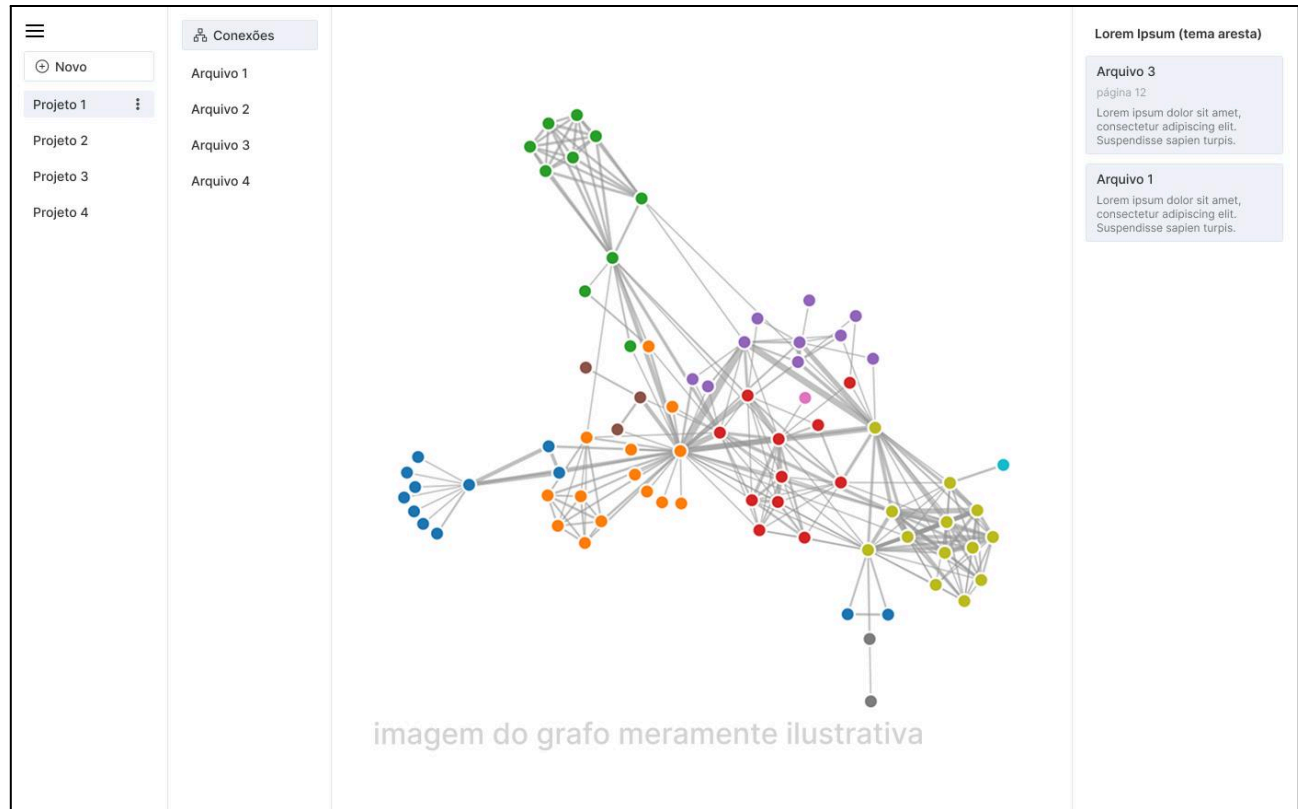


Figura 24. Tela de conexões (grafo)

A Figura 24 apresenta o *wireframe* da funcionalidade mais inovadora do Cognita. A área central exibe o grafo de conhecimento gerado automaticamente, com os nós representando conceitos e as arestas, suas relações. O painel esquerdo mantém a navegação entre projetos. Crucialmente, o painel direito, intitulado "Contexto", exibe informações detalhadas sobre o nó ou a conexão selecionada no grafo, como o trecho de texto original, o nome do arquivo e a página, cumprindo o requisito de fornecer contexto sem que o usuário precise sair da visualização.

5. Glossário e Modelos de Dados

Esta seção apresenta o glossário de termos do sistema e detalha o modelo de dados que define a estrutura de persistência da informação.

5.1 Glossário

O glossário a seguir define os principais atributos de dados com os quais o usuário interage, seja fornecendo-os como entrada ou visualizando-os como saída do sistema. O objetivo é padronizar a terminologia e esclarecer o formato e a descrição de cada campo reconhecido pelo usuário, conforme apresentado na Tabela 7.

Atributo	Formato	Descrição
Nome do Usuário	Texto	Nome completo do usuário, utilizado para identificação e personalização da conta. É uma entrada de dados durante o cadastro.
Email do Usuário	Email	Endereço de e-mail do usuário, que serve como identificador único para login e para comunicação do sistema.
Senha	Senha	Credencial de segurança definida pelo usuário para acesso à sua conta. O sistema armazena este dado de forma hash criptográfico.
Nome do Projeto	Texto	Nome descritivo que o usuário atribui a um projeto para agrupar documentos e notas relacionados a um mesmo tema ou matéria.
Data de Criação do Projeto	Data	Data em que o projeto foi criado. É uma saída de dados gerada automaticamente pelo sistema e pode ser exibida na interface.
Arquivo PDF	Arquivo (.pdf)	O arquivo em formato PDF que o usuário seleciona em seu dispositivo para fazer upload para a sua biblioteca no Cognita.
Título do Documento	Texto	O nome do arquivo PDF que foi importado, exibido na lista de documentos de um projeto.
Status do Documento	Texto (Enum)	Indicação visual do estado de processamento de um documento (ex: "Processando", "Indexado", "Falha"). É uma saída de dados do sistema.
Conteúdo da Anotação	Texto / Vetor	O conteúdo da anotação criada pelo usuário. Pode ser um texto digitado, um destaque sobre um PDF ou os dados vetoriais de um traço manuscrito.
Cor da Anotação	Cor (Hex)	A cor escolhida pelo usuário para uma ferramenta de anotação, como um marca-texto ou uma caneta.
Espessura da Anotação	Numérico	A espessura do traço escolhida pelo usuário para uma ferramenta de anotação, como a caneta.
Termo de Busca	Texto	A palavra ou frase que o usuário digita no campo de busca para encontrar informações em sua base de conhecimento.
Lista de Resultados	Lista	A lista de trechos de texto e suas fontes (documento, página) que o sistema exibe como resultado de uma busca.

Rótulo do Conceito	Texto	O nome da entidade ou conceito (ex: "UNESCO") extraído pelo sistema e exibido como um "nó" na visualização do grafo.
Contexto da Conexão	Texto	O trecho de texto original de um documento ou anotação que gerou uma menção a um conceito, exibido ao interagir com o grafo.

Tabela 7. Glossário

5.2 Modelo de Dados – Property Graph Model

Dada a escolha arquitetural de utilizar um banco de dados de grafo nativo (Neo4j) para a persistência dos dados do servidor, um Diagrama de Entidade-Relacionamento (DER) tradicional não é o modelo mais adequado. Em seu lugar, utilizamos o *Property Graph Model*, que é o modelo conceitual nativo para bancos de dados de grafo e representa a estrutura de dados de forma mais fiel à sua implementação.

O *Property Graph Model* descreve os dados em termos de Nós, Relações e Propriedades:

- Nós (*Nodes*): Representam as entidades do sistema. Cada nó pode ter um ou mais Rótulos (*Labels*) que definem seu tipo ou papel no grafo.
- Relações (*Relationships*): Representam as conexões semânticas e direcionadas entre nós. Cada relação possui um único Tipo que descreve a natureza da conexão.
- Propriedades (*Properties*): São pares de chave-valor que armazenam os dados e podem ser atribuídos tanto aos Nós quanto às Relações.

A Figura 25 ilustra o *Property Graph Model* para o sistema Cognita, mostrando como as classes definidas no Diagrama de Classes são mapeadas para nós e como seus relacionamentos são representados.

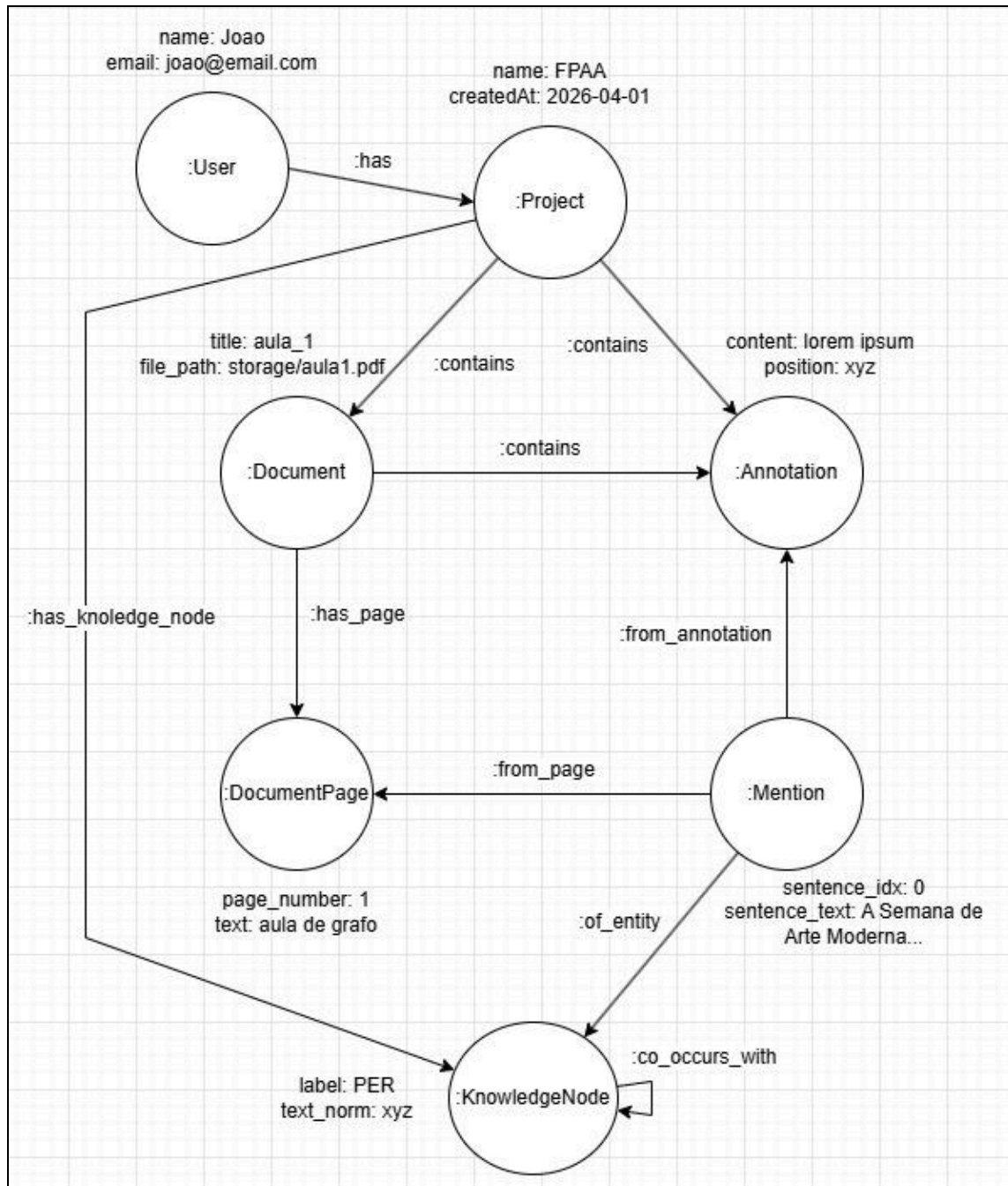


Figura 25. Property Graph Model

6. Casos de Teste

Esta seção descreve a estratégia de validação e verificação do *software* Cognita, dividida em testes de aceitação (focados no usuário e requisitos) e testes de integração (focados na arquitetura e comunicação entre componentes).

6.1 Plano de Teste de Aceitação

O objetivo dos testes de aceitação é validar se as funcionalidades desenvolvidas atendem às necessidades do usuário final conforme definido no Documento de Visão. A Tabela 8 apresenta os casos de teste mapeados para as principais necessidades do sistema.

Necessidade (Doc. Visão)	ID	Descrição do Caso de Teste	Entrada de Dados (Ação)	Saída Esperada / Comportamento
1. Gestão da Biblioteca	TA-01	<i>Upload</i> de arquivo PDF válido	Selecionar um arquivo qualquer com texto e confirmar <i>upload</i> .	O sistema deve exibir o arquivo na lista "Meus Documentos" com status "Processando".
	TA-02	Tentativa de <i>upload</i> de formato inválido	Selecionar arquivo "Imagem.png" ou "Texto.docx".	O sistema deve exibir mensagem de erro: "Formato não suportado. Apenas PDF." e não iniciar o <i>upload</i> .
	TA-03	Visualização de documento processado	Clicar em um documento com status "Indexado".	O documento deve abrir no visualizador pronto para leitura.
2. Anotação de Conteúdo	TA-04	Criação de anotação manuscrita	Usar a <i>stylus</i> para desenhar um círculo vermelho sobre um parágrafo.	O traço vermelho deve ser renderizado instantaneamente sobre o PDF e persistir no <i>backend</i> .
	TA-05	Remoção de anotação	Selecionar a ferramenta borracha e passar sobre um traço existente.	O traço deve ser removido da visualização e do sistema.
3. Busca Global	TA-06	Busca por termo presente no texto	Digitar "Redes Neurais" na barra de busca global.	O sistema deve listar todos os documentos e anotações que contêm o termo, indicando a página da ocorrência.
	TA-07	Busca por termo em	Digitar uma palavra que foi escrita à mão	O sistema deve encontrar a anotação manuscrita

		anotação manuscrita	(ex: "Revisar") em uma nota.	(validando o HCR) e levar o usuário até ela.
	TA-08	Navegação para resultado da busca	Clicar no 2º item da lista de resultados.	O sistema deve abrir o documento correspondente e rolar automaticamente até a página onde o termo foi encontrado.
4. Conexão de Conhecimento (Grafo)	TA-09	Visualização do Grafo de Conhecimento	Acessar a aba "Grafo" de um projeto.	O sistema deve renderizar uma rede de nós (conceitos) e arestas (relações) interativos.
	TA-10	Exploração de Conexão	Clicar na linha (aresta) que liga "IA" e "Ética".	Um painel lateral deve abrir exibindo o trecho original do texto que fundamenta essa conexão ("Contexto").

Tabela 8. Casos de Teste de Aceitação

6.2 Plano de Teste de Integração

Os testes de integração visam garantir que os subsistemas e serviços externos do Cognita comuniquem-se corretamente. Dada a arquitetura distribuída, o foco é validar as interfaces entre o Cliente, o Servidor e os Serviços de Nuvem. A Tabela 9 apresenta os casos de teste mapeados para as principais necessidades do sistema.

Estratégia de Teste: Será utilizada uma estratégia Incremental Ascendente (*Bottom-Up*) focada em serviços:

- Isolamento: os testes automatizados que tocam o banco de grafo utilizam uma instância Neo4j de teste dedicada, garantindo que as *queries* Cypher rodem em um ambiente controlado e descartável.
- *Mocks*: para serviços pagos ou lentos, são utilizados *Mocks* (simuladores) no ambiente de desenvolvimento, permitindo validar a lógica de tratamento das respostas sem gerar custos.
- Integração Cliente-Servidor: testes de API (contrato) validam se os endpoints REST aceitam e retornam os dados no formato JSON esperado pelo aplicativo React Native.

Interface	Componentes Envolvidos	ID Teste	Cenário de Teste	Critério de Sucesso
-----------	------------------------	----------	------------------	---------------------

REST API	Cliente (<i>Mobile</i>) e Servidor (<i>Backend</i>)	TI-01	Contrato REST de Criação de Anotação: o cliente envia uma anotação (JSON) ao <i>endpoint</i> POST do servidor.	O servidor deve aceitar o <i>payload</i> , retornar status 200/201 no formato esperado e disparar a operação de persistência da anotação. A integração com o Neo4j real é coberta separadamente pelo TI-03.
Storage API	Servidor e Armazenamento de Arquivo	TI-02	Upload de Arquivo no Armazenamento de Objetos: o serviço de biblioteca envia um stream de bytes do PDF ao componente de Storage (MinIO).	O serviço deve receber uma URL pública/assinada do arquivo, e o conteúdo deve estar acessível por essa URL. Previsto como evolução nesta versão; será implementado no mesmo padrão do TI-03.
Bolt Protocol	Servidor e Neo4j	TI-03	Persistência de Grafo: O processador de conteúdo tenta salvar 50 nós e 100 relações de uma vez.	A transação deve ser commitada no Neo4j e uma <i>query</i> de contagem deve retornar os números exatos inseridos.
Vision API	Servidor e API de Visão	TI-04	Fluxo de OCR: O servidor envia uma imagem para a API de Visão (simulada via <i>mock</i>).	O sistema deve tratar corretamente a resposta JSON da API, extraindo o bloco de texto ("fullTextAnnotation") e ignorando metadados irrelevantes.

Tabela 9. Casos de Teste de Integração

6.3 Avaliação da Qualidade do Grafo de Conhecimento

O grafo de conhecimento é construído em duas etapas de naturezas distintas, o que exige estratégias de avaliação diferentes:

- O Reconhecimento de Entidades Nomeadas (NER) é realizado por um modelo de PLN de natureza probabilística (spaCy pt_core_news_lg). Por ser estatístico, está sujeito a erros: pode deixar de identificar uma entidade, classificá-la no tipo incorreto (Pessoa, Local ou

Organização) ou delimitar mal o trecho reconhecido. É a etapa que concentra a necessidade de avaliação de qualidade.

- A Extração de Relações (RE) é feita por co-ocorrência, uma abordagem determinística: duas entidades são conectadas quando aparecem na mesma sentença, e o peso da aresta corresponde ao número de sentenças em que isso ocorre. Como não há geração de texto nem inferência de um modelo generativo, não existe risco de "alucinação" (relações inventadas e ausentes do texto-fonte). O ponto de atenção aqui não é a alucinação, mas o ruído: ligações entre entidades que dividem uma frase de forma acidental, sem relação semântica real. Esse ruído é mitigado pelo próprio peso da aresta, que pode ser usado como filtro; conexões com baixa co-ocorrência tendem a ser menos confiáveis.

A qualidade do resultado não será medida apenas pela execução bem-sucedida do código, mas pela aderência do grafo gerado ao conteúdo das fontes. Para isso, será realizado um teste de validação manual comparativo:

- Conjunto de Validação (*Gold Standard*): será selecionada uma amostra de 3 documentos acadêmicos de domínios variados, anotados manualmente para definir as entidades que deveriam ser reconhecidas e os pares de entidades que de fato co-ocorrem de forma significativa. A partir dessa referência, avaliam-se:
 - no NER: a precisão (proporção das entidades extraídas que estão corretas) e a revocação (proporção das entidades relevantes que o modelo conseguiu capturar);
 - nas arestas de co-ocorrência: a precisão das ligações geradas — isto é, quantas das conexões correspondem a uma relação real entre os conceitos, em oposição a co-ocorrências por acaso.

7. Cronograma e Processo de Implementação

Esta seção detalha o plano de gerenciamento do projeto, incluindo o cronograma de desenvolvimento e os processos que serão adotados para a implantação e entrega do sistema Cognita.

7.1 Cronograma do Projeto

O desenvolvimento do sistema será gerenciado utilizando uma abordagem ágil, baseada em ciclos de *sprint* com duração de duas semanas (quinzenais). O cronograma total de desenvolvimento está planejado para ocorrer ao longo de aproximadamente quatro meses, com início em fevereiro de 2026 e conclusão na primeira semana de junho de 2026, totalizando 9 *sprints* de desenvolvimento.

O foco de cada *sprint* está alinhado com a entrega de valor incremental, priorizando a construção das funcionalidades centrais da arquitetura antes das funcionalidades de IA mais complexas. A Tabela 10 detalha o plano de entregas para cada *sprint*.

Sprint	Período (2026)	Foco Principal	Entregáveis Chave
1	01/Fev – 15/Fev	Configuração e Arquitetura Base	- Configuração dos ambientes (Local, Servidor). - Setup do projeto React Native. - Setup da API do Servidor e do banco de dados Neo4j.
2	16/Fev – 28/Mar	Modelo de Dados e Autenticação	- Implementação do modelo no Neo4j. - Implementação do backend de autenticação (Cadastro, Login). - Telas de Login e Cadastro no cliente.
3	01/Mar – 15/Mar	Cliente e Gestão de Projetos	- Telas de gerenciamento de Projetos. - Persistência via API de usuários e projetos.
4	16/Mar – 31/Mar	<i>Pipeline de Upload e Processamento</i>	- Implementação do <i>upload</i> de arquivos. - Integração com o Armazenamento de arquivo. - <i>Backend</i>: Gatilho para o <i>pipeline</i> de processamento.
5	01/Abr – 15/Abr	Anotação Manuscrita	- Implementação da tela de anotação com WebView. - Captura de traço da stylus. - Persistência via API das anotações.
6	16/Abr – 30/Abr	<i>Pipeline de IA (Parte 1): OCR/HCR</i>	- Integração do <i>backend</i> com a API de Visão. - <i>Backend</i>: Processamento de PDFs e imagens de notas. - Armazenamento do texto extraído.
7	1/Mai – 15/Mai	Pipeline de IA (Parte 2): NER e RE	- Integração dos modelos spaCy e co-ocorrência. - <i>Backend</i>: Extração de entidades e relações do texto processado.
8	16/Mai – 31/Mai	Persistência e Visualização do Grafo	- Armazenamento dos nós e arestas no Neo4j. - Implementação da tela de visualização do grafo. - Validação visual do <i>pipeline</i> de IA de

			ponta a ponta.
9	1/Jun – 15/Jun	Busca Global e Teste de Aceite	- Implementação da funcionalidade de Busca Global. - Execução dos casos de testes. - Correção de <i>bugs</i> e refatoração final.

Tabela 10. Cronograma

7.2 Processo de Implementação

O processo de desenvolvimento do sistema Cognita foi conduzido sob uma metodologia ágil e incremental, organizada em *sprints* quinzenais, conforme detalhado na seção 7.1. Este ciclo permitiu a entrega contínua de valor e a rápida adaptação a mudanças de escopo e desafios técnicos identificados.

Práticas Adotadas

- Controle de Versão: Foi adotado o Git para controle de versão, utilizando a plataforma GitHub como repositório central.
- Testes Automatizados: A garantia de qualidade foi suportada por testes automatizados, executados de forma local e manual. O *backend* utiliza o *framework* pytest para testes unitários e um teste de integração real contra o banco de dados Neo4j. O lado *mobile* (React Native) utiliza Jest combinado com a React Native Testing Library.

Artefatos Produzidos

- Código-Fonte: Os códigos completos do *backend* (FastAPI/Python) e da aplicação *mobile* (React Native/Expo), incluindo seus respectivos pacotes de testes automatizados.
- Documentação: Arquivos README detalhados na raiz do projeto com instruções para instalação e execução, além de READMEs específicos para cada componente, visando facilitar a manutenção.

Evoluções Previstas

A implantação do sistema, atualmente realizada de forma manual, prevê como próximos passos a adoção de práticas completas de Integração Contínua (CI/CD) e o desenvolvimento de um *pipeline* para implantação automatizada.

8. Post-Mortem

Esta seção apresenta uma análise crítica e reflexiva sobre o projeto Cognita, detalhando as conquistas alcançadas, os desafios técnicos enfrentados e as decisões de *design* que resultaram em limitações. O objetivo é extrair aprendizados práticos da experiência de desenvolvimento e traçar um caminho claro para a evolução futura do sistema.

8.1 Conquistas e Aprendizados

O projeto demonstrou a viabilidade de transformar material de estudo em uma base de conhecimento navegável, superando desafios complexos de integração e processamento.

- Grafo de Conhecimento Automático: A plataforma extrai conceitos automaticamente via Reconhecimento de Entidades Nomeadas (NER) e estabelece conexões por co-ocorrência. O grafo resultante é navegável, com a crucial funcionalidade de *mentions* que levam o usuário de volta à frase exata na fonte original.
- Pipeline Assíncrono Desacoplado: A arquitetura de ProcessadorConteúdo assíncrono isolou o processamento pesado de OCR/HCR e a construção do grafo (Neo4j) da API principal, garantindo que a interface do usuário permaneça responsiva durante operações demoradas.
- Anotação Fluida em Tablet: Foi implementado um *canvas* em WebView que permite anotação manuscrita e digitada, com renderização local instantânea do traço, fornecendo uma experiência de uso otimizada para tablets na leitura de PDFs.
- Cobertura de Testes: Foram implementados testes automatizados em ambas as camadas: o *backend* com pytest (incluindo teste de integração real com Neo4j) e o *mobile* com Jest e React Native Testing Library.
- Aprendizado Prático: O desenvolvimento proporcionou a experiência prática com bancos de dados de grafos (Neo4j/Cypher), o uso de ferramentas de Processamento de Linguagem Natural (PLN) em português e a implementação de uma arquitetura Cliente-Servidor em camadas.

8.2 Experiências Negativas e Limitações

As dificuldades encontradas no desenvolvimento resultaram em mudanças arquiteturais significativas, que impuseram limitações ao produto final.

- Abandono do *Offline-First*: O projeto foi originalmente planejado para ser *offline-first* (com banco local WatermelonDB), mas a implementação foi abandonada em favor de uma arquitetura REST pura, tornando o cliente dependente de conexão. Esta é uma limitação relevante para o público-alvo, que pode estudar em ambientes com conectividade instável.
- Extração de Relações (RE) por Co-ocorrência: Embora seja uma abordagem determinística, o método de co-ocorrência gera arestas sem rótulo semântico, podendo resultar em ligações fortuitas (ruído) entre entidades que apenas compartilham a mesma frase, mitigado apenas pelo peso da aresta.
- Qualidade do NER em Português: O modelo adotado, spaCy pt_core_news_lg, é probabilístico e pode gerar entidades incorretas ou deixar de detectar algumas, impactando diretamente a qualidade e a estrutura do grafo. A escolha por ele, ciente dessa limitação, foi pragmática: roda localmente, sem custo nem latência de rede, já vem pré-treinado para o português e dispensa GPU ou fine-tuning; características que se encaixam nas restrições de orçamento e infraestrutura do projeto. Alternativas com qualidade superior (modelos baseados em BERT/transformers) exigiriam recursos computacionais incompatíveis com o escopo atual; seu uso fica registrado como evolução futura.

- Dependência de Serviço Externo (Google Vision): A utilização do Google Cloud Vision para OCR/HCR, embora confiável, impõe restrições de custo e limite do *free-tier*.

8.3 Segurança, Conformidade e Restrições

A segurança dos dados e a conformidade legal foram consideradas no *design* do sistema, com foco na arquitetura agnóstica e na proteção das credenciais do usuário.

- Autenticação: O sistema utiliza *tokens* JWT (HS256) assinados com chave secreta para autenticação.
- Armazenamento de Senhas: As senhas dos usuários são armazenadas utilizando *hash* bcrypt com *salt* gerado por gensalt.
- Armazenamento de Arquivos: Os arquivos no MinIO são acessados por URLs assinadas, garantindo acesso restrito.
- Ponto de Atenção LGPD: PDFs e imagens de anotação são enviados à Google Vision (um processador externo) para OCR/HCR, exigindo uma avaliação detalhada sobre consentimento e tratamento de dados pessoais.
- Arquitetura Agnóstica: O *design* do sistema foi concebido para ser agnóstico a fornecedor, permitindo a substituição de componentes externos como Vision e MinIO com impacto mínimo na lógica central.

8.4 Integração com Serviços Externos

Os serviços externos foram selecionados para atender aos requisitos de processamento e persistência, respeitando as restrições orçamentárias.

- Google Vision (OCR/HCR): O serviço se mostrou confiável, mas o uso está sujeito aos limites de custo da *free-tier*. Recomenda-se o tratamento adequado de erros e limites, além de uma futura implementação de *cache*.
- Neo4j e MinIO: O banco de grafo (Bolt) e o armazenamento de objetos (S3-compatível) são fundamentais à arquitetura e são substituíveis, reforçando o princípio de design agnóstico a fornecedor.

8.5 Lições Aprendidas e Próximos Passos

A experiência de desenvolvimento destacou prioridades para o público-alvo e a necessidade de aprimorar as capacidades de PLN do sistema.

- Persistência *Offline* Real: A implementação de um modo *offline* com fila de sincronização é essencial para a experiência do usuário e deve ser priorizada.
- Extração de Relações Semântica: A utilização de um modelo dedicado ou LLM para Extração de Relações permitiria rotular as arestas, fornecendo significado explícito às conexões no grafo.

- Melhoria do NER: É crucial melhorar a qualidade do Reconhecimento de Entidades (NER) por meio de um modelo específico ou *fine-tuning*.
- CI/CD e *Deploy* Automatizado: O desenvolvimento futuro deve incluir a criação de um *pipeline* de CI/CD e a automação de *deploy*.

8.6 Melhoria Contínua / Recomendações Futuras

- Implementar o modo *offline* com sincronização de dados.
- Desenvolver a Extração de Relações semântica para rotular as arestas do grafo.
- Adicionar filtro do grafo por tipo de entidade e métricas de qualidade do grafo.
- Configurar CI/CD e *deploy* automatizado.
- Realizar validação de usabilidade com usuários reais.

9. Conclusão

O sistema Cognita cumpriu o seu propósito central, transformando material de estudo em um grafo de conhecimento navegável, interconectando conceitos e possibilitando a rastreabilidade da informação até a fonte original. A arquitetura demonstrou robustez na separação de responsabilidades, evidenciada pelo *pipeline* assíncrono e pela implementação de testes automatizados em ambas as camadas.

A honesta avaliação do projeto, contudo, revela limitações cruciais, como o abandono do modo *offline*, a Extração de Relações baseada em co-ocorrência e a dependência da qualidade do NER e do serviço externo Google Vision. Tais pontos, alinhados às restrições do Documento de Visão, servem de base para um caminho sólido de evolução, onde a priorização de uma persistência local robusta e o aprimoramento semântico do grafo se configuram como os próximos passos essenciais para a entrega de valor completo ao público-alvo.